```
package tacos.email;

import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.stereotype.Component;
import lombok.Data;

@Data
@ConfigurationProperties(prefix = "tacocloud.api")
@Component
public class ApiProperties {
  private String url;
}
```

And in application.yml, the URL for the Taco Cloud API might be configured like this:

```
tacocloud:
  api:
    url: http://localhost:8080/orders/fromEmail
```

To make `RestTemplate` available in the project so that it can be injected into `OrderSubmitMessageHandler`, you need to add the Spring Boot web starter to the project build like so:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Although this makes `RestTemplate` available in the classpath, it also triggers autoconfiguration for Spring MVC. As a standalone Spring Integration flow, the application doesn't need Spring MVC or even the embedded Tomcat that autoconfiguration provides. Therefore, you should disable Spring MVC autoconfiguration with the following entry in application.yml:

```
spring:
  main:
    web-application-type: none
```

The `spring.main.web-application-type` property can be set to either `servlet`, `reactive`, or `none`. When Spring MVC is in the classpath, autoconfiguration sets its value to `servlet`. But here you override it to `none` so that Spring MVC and Tomcat won't be autoconfigured. (We'll talk more about what it means for an application to be a reactive web application in chapter 12.)

Summary

- Spring Integration enables the definition of flows through which data can be processed as it enters or leaves an application.
- Integration flows can be defined in XML, Java, or using a succinct Java DSL configuration style.
- Message gateways and channel adapters act as entry and exit points of an integration flow.
- Messages can be transformed, split, aggregated, routed, and processed by service activators in the course of a flow.
- Message channels connect the components of an integration flow.

Part 3

Reactive Spring

In part 3, we'll explore the support for reactive programming in Spring. Chapter 11 discusses the essentials of reactive programming with Project Reactor, the reactive programming library that underpins Spring's reactive features. We'll then look at some of Reactor's most useful reactive operations. In chapter 12, we'll revisit REST API development, introducing Spring WebFlux, a web framework that borrows much from Spring MVC while offering a new reactive model for web development. Chapter 13 takes a look at writing reactive data persistence with Spring Data to read and write data to Cassandra and Mongo databases. Chapter 14 rounds out part 3 by looking at RSocket, a communication protocol that enables a reactive alternative to HTTP.

11 Introducing Reactor

This chapter covers

- Understanding reactive programming
- Project Reactor
- Operating on data reactively

Have you ever held a subscription for a newspaper or a magazine? The internet has certainly taken a bite out of the subscriber base of traditional publications, but there was a time when a newspaper subscription was one of the best ways to keep up with the events of the day. You could count on a fresh delivery of current events every morning, to read during breakfast or on the way to work.

Now suppose that if, after paying for your subscription, several days go by and no papers have been delivered. A few more days go by, and you call the newspaper sales office to ask why you haven't yet received your daily paper. Imagine your surprise if they explain, "You paid for a full year of newspapers. The year hasn't completed yet. You'll certainly receive them all once the full year of newspapers is ready."

Thankfully, that's not at all how subscriptions work. Newspapers have a certain timeliness to them. They're delivered as quickly as possible after publication so that they can be read while their content is still fresh. Moreover, as you're reading the

latest issue, newspaper reporters are writing new stories for future editions, and the presses are fired up producing the next edition—all in parallel.

As we develop application code, we can write two styles of code—imperative and reactive, which are described as follows:

- *Imperative* code is a lot like that absurd hypothetical newspaper subscription. It's a serial set of tasks, each running one at a time, each after the previous task. Data is processed in bulk and can't be handed over to the next task until the previous task has completed its work on the bulk of data.
- *Reactive* code is a lot like a real newspaper subscription. A set of tasks is defined to process data, but those tasks can run in parallel. Each task can process subsets of the data, handing it off to the next task in line while it continues to work on another subset of the data.

In this chapter, we're going to step away from the Taco Cloud application temporarily to explore Project Reactor (https://projectreactor.io/). Reactor is a library for reactive programming that's part of the Spring family of projects. And because it serves as the foundation of Spring's support for reactive programming, it's important that you understand Reactor before we look at building reactive controllers and repositories with Spring. Before we start working with Reactor, though, let's quickly examine the essentials of reactive programming.

11.1 Understanding reactive programming

Reactive programming is a paradigm that's an alternative to imperative programming. This alternative exists because reactive programming addresses a limitation in imperative programming. By understanding these limitations, you can better grasp the benefits of the reactive model.

> **NOTE** Reactive programming isn't a silver bullet. In no way should you infer from this chapter or any other discussion of reactive programming that imperative programming is evil and that reactive programming is your savior. Like anything you learn as a developer, reactive programming is a perfect fit in some use cases, and it's ill-fitted in others. An ounce of pragmatism is advised.

If you're like me and many developers, you cut your programming teeth with imperative programming. There's a good chance that most (or all) of the code you write today is still imperative in nature. Imperative programming is intuitive enough that young students are learning it with ease in their school's STEM programs, and it's powerful enough that it makes up the bulk of code that drives the largest enterprises.

The idea is simple: you write code as a list of instructions to be followed, one at a time, in the order that they're encountered. A task is performed and the program waits for it to complete before moving on to the next task. At each step along the way, the data that's to be processed must be fully available so that it can be processed as a whole.

This is fine . . . until it isn't. While a task is being performed—and especially if it's an I/O task, such as writing data to a database or fetching data from a remote server—the thread that invoked that task is blocked, unable to do anything else until the task completes. To put it bluntly, blocked threads are wasteful.

Most programming languages, including Java, support concurrent programming. It's fairly easy to fire up another thread in Java and send it on its way to perform some work while the invoking thread carries on with something else. But although it's easy to create threads, those threads are likely to end up blocked themselves. Managing concurrency in multiple threads is challenging. More threads mean more complexity.

In contrast, reactive programming is functional and declarative in nature. Rather than describe a set of steps that are to be performed sequentially, reactive programming involves describing a pipeline or stream through which data flows. Rather than requiring the data to be available and processed as a whole, a reactive stream processes data as it becomes available. In fact, the incoming data may be endless (a constant stream of a location's real-time temperature data, for instance).

> **NOTE** If you're new to functional programming in Java, you may want to have a look at *Functional Programming in Java* by Pierre-Yves Saumont (Manning, 2017), or *Grokking Functional Programming* by Michał Płachta (Manning, 2021).

To apply a real-world analogy, consider imperative programming as a water balloon and reactive programming as a garden hose. Both are suitable ways to surprise and soak an unsuspecting friend on a hot summer day. But they differ in their execution style as follows:

- A water balloon carries its payload all at once, soaking its intended target at the moment of impact. The water balloon has a finite capacity, however, and if you wish to soak more people (or the same person to a greater extent), your only choice is to scale up by increasing the number of water balloons.
- A garden hose carries its payload as a stream of water that flows from the spigot to the nozzle. The garden hose's capacity may be finite at any given point in time, but it's unlimited over the course of a water battle. As long as water is entering the hose from the spigot, it will continue to flow through the hose and spray out of the nozzle. The same garden hose is easily scalable to soak as many friends as you wish.

There's nothing inherently wrong with water balloons (or imperative programming), but the person holding the garden hose (or applying reactive programming) has an advantage in regard to scalability and performance.

11.1.1 Defining Reactive Streams

Reactive Streams is an initiative started in late 2013 by engineers from Netflix, Lightbend, and Pivotal (the company behind Spring). Reactive Streams aims to provide a standard for asynchronous stream processing with nonblocking backpressure.

We've already touched on the asynchronous trait of reactive programming; it's what enables us to perform tasks in parallel to achieve greater scalability. Backpressure is a means by which consumers of data can avoid being overwhelmed by an overly fast data source, by establishing limits on how much they're willing to handle.

Java streams vs. Reactive Streams

There's a lot of similarity between Java streams and Reactive Streams. To start with, they both have the word *streams* in their names. They also both provide a functional API for working with data. In fact, as you'll see later when we look at Reactor, they even share many of the same operations.

Java streams, however, are typically synchronous and work with a finite set of data. They're essentially a means of iterating over a collection with functions.

Reactive Streams support asynchronous processing of datasets of any size, including infinite datasets. They process data in real time, as it becomes available, with backpressure to avoid overwhelming their consumers.

On the other hand, JDK 9's Flow APIs correspond to Reactive Streams. The `Flow.Publisher`, `Flow.Subscriber`, `Flow.Subscription`, and `Flow.Processor` types in JDK 9 map directly to `Publisher`, `Subscriber`, `Subscription`, and `Processor` in Reactive Streams. That said, JDK 9's Flow APIs are not an actual implementation of Reactive Streams.

The Reactive Streams specification can be summed up by four interface definitions: `Publisher`, `Subscriber`, `Subscription`, and `Processor`. A `Publisher` produces data that it sends to a `Subscriber` per a `Subscription`. The `Publisher` interface declares a single method, `subscribe()`, through which a `Subscriber` can subscribe to the `Publisher`, as shown here:

```
public interface Publisher<T> {
  void subscribe(Subscriber<? super T> subscriber);
}
```

Once a `Subscriber` has subscribed, it can receive events from the `Publisher`. Those events are sent via methods on the `Subscriber` interface as follows:

```
public interface Subscriber<T> {
  void onSubscribe(Subscription sub);
  void onNext(T item);
  void onError(Throwable ex);
  void onComplete();
}
```

The first event that the `Subscriber` will receive is through a call to `onSubscribe()`. When the `Publisher` calls `onSubscribe()`, it passes a `Subscription` object to the `Subscriber`. It's through the `Subscription` that the `Subscriber` can manage its subscription, as shown next:

```
public interface Subscription {
  void request(long n);
  void cancel();
}
```

The `Subscriber` can call `request()` to request that data be sent, or it can call `cancel()` to indicate that it's no longer interested in receiving data and is canceling the subscription. When calling `request()`, the `Subscriber` passes in a `long` value to indicate how many data items it's willing to accept. This is where backpressure comes in, preventing the `Publisher` from sending more data than the `Subscriber` is able to handle. After the `Publisher` has sent as many items as were requested, the `Subscriber` can call `request()` again to request more.

Once the `Subscriber` has requested data, the data starts flowing through the stream. For every item that's published by the `Publisher`, the `onNext()` method will be called to deliver the data to the `Subscriber`. If there are any errors, `onError()` is called. If the `Publisher` has no more data to send and isn't going to produce any more data, it will call `onComplete()` to tell the `Subscriber` that it's out of business.

As for the `Processor` interface, it's a combination of `Subscriber` and `Publisher`, as shown here:

```
public interface Processor<T, R>
         extends Subscriber<T>, Publisher<R> {}
```

As a `Subscriber`, a `Processor` will receive data and process it in some way. Then it will switch hats and act as a `Publisher` to publish the results to its `Subscribers`.

As you can see, the Reactive Streams specification is rather straightforward. It's fairly easy to see how you could build up a data processing pipeline that starts with a `Publisher`, pumps data through zero or more `Processors`, and then drops the final results off to a `Subscriber`.

What the Reactive Streams interfaces don't lend themselves to, however, is composing such a stream in a functional way. Project Reactor is an implementation of the Reactive Streams specification that provides a functional API for composing Reactive Streams. As you'll see in the following chapters, Reactor is the foundation for Spring's reactive programming model. In the remainder of this chapter, we're going to explore (and, dare I say, have a lot of fun with) Project Reactor.

11.2 Getting started with Reactor

Reactive programming requires us to think in a very different way from imperative programming. Rather than describe a set of steps to be taken, reactive programming means building a pipeline through which data will flow. As data passes through the pipeline, it can be altered or used in some way.

For example, suppose you want to take a person's name, change all of the letters to uppercase, use it to create a greeting message, and then finally print it. In an imperative programming model, the code would look something like this:

```
String name = "Craig";
String capitalName = name.toUpperCase();
String greeting = "Hello, " + capitalName + "!";
System.out.println(greeting);
```

In the imperative model, each line of code performs a step, one right after the other, and definitely in the same thread. Each step blocks the executing thread from moving to the next step until complete.

In contrast, functional, reactive code could achieve the same thing like this:

```
Mono.just("Craig")
    .map(n -> n.toUpperCase())
    .map(cn -> "Hello, " + cn + "!")
    .subscribe(System.out::println);
```

Don't worry too much about the details of this example; we'll talk all about the `just()`, `map()`, and `subscribe()` operations soon enough. For now, it's important to understand that although the reactive example still seems to follow a step-by-step model, it's really a pipeline that data flows through. At each phase of the pipeline, the data is tweaked somehow, but no assumption can be made about which thread any of the operations are performed on. They may be the same thread . . . or they may not be.

The `Mono` in the example is one of Reactor's two core types. `Flux` is the other. Both are implementations of Reactive Streams' `Publisher`. A `Flux` represents a pipeline of zero, one, or many (potentially infinite) data items. A `Mono` is a specialized reactive type that's optimized for when the dataset is known to have no more than one data item.

Reactor vs. RxJava (ReactiveX)

If you're already familiar with RxJava or ReactiveX, you may be thinking that `Mono` and `Flux` sound a lot like `Observable` and `Single`. In fact, they're approximately equivalent semantically. They even offer many of the same operations.

Although we focus on Reactor in this book, you may be happy to know that it's possible to covert between Reactor and RxJava types. Moreover, as you'll see in the following chapters, Spring can also work with RxJava types.

The previous example actually contains three `Mono` objects. The `just()` operation creates the first one. When the `Mono` emits a value, that value is given to the `map()` operation to be capitalized and used to create another `Mono`. When the second `Mono` publishes its data, it's given to the second `map()` operation to do some `String` concatenation, the results of which are used to create the third `Mono`. Finally, the call to `subscribe()` subscribes to the `Mono`, receives the data, and prints it.

11.2.1 Diagramming reactive flows

Reactive flows are often illustrated with marble diagrams. Marble diagrams, in their simplest form, depict a timeline of data as it flows through a `Flux` or `Mono` at the top, an operation in the middle, and the timeline of the resulting `Flux` or `Mono` at the bottom. Figure 11.1 shows a marble diagram template for a `Flux`. As you can see, as data flows through the original `Flux`, it's processed through some operation, resulting in a new `Flux` through which the processed data flows.

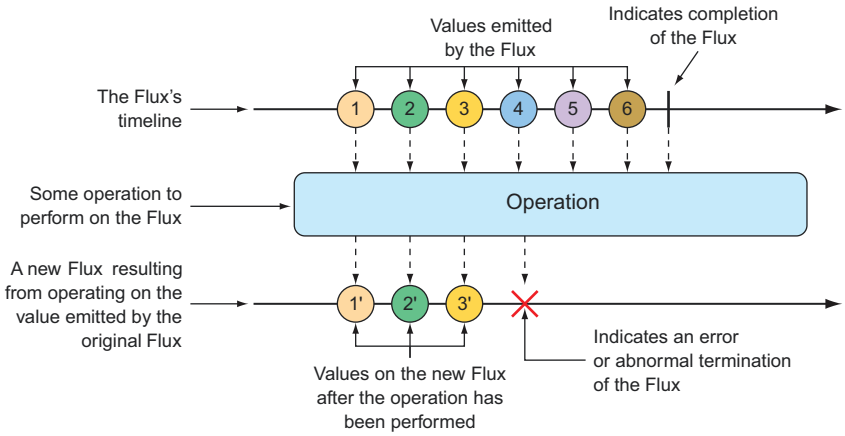

Figure 11.1 Marble diagram illustrating the basic flow of a `Flux`

Figure 11.2 shows a similar marble diagram, but for a `Mono`. As you can see, the key difference is that a `Mono` will have either zero or one data item, or an error.

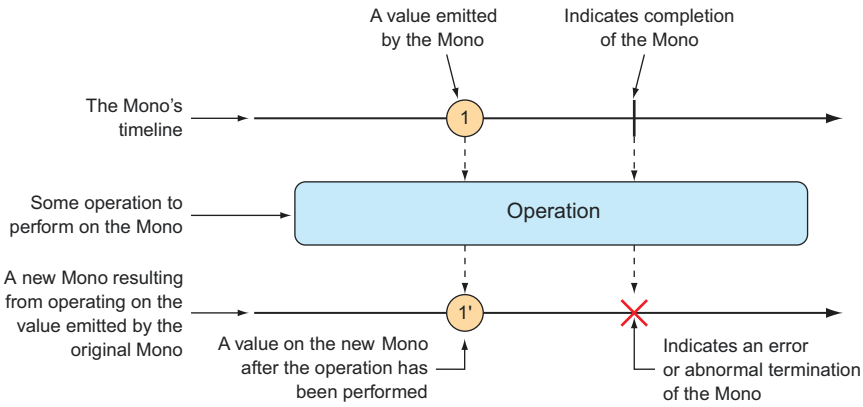

Figure 11.2 Marble diagram illustrating the basic flow of a `Mono`

In section 11.3, we'll explore many operations supported by `Flux` and `Mono`, and we'll use marble diagrams to visualize how they work.

11.2.2 Adding Reactor dependencies

To get started with Reactor, add the following dependency to the project build:

```
<dependency>
  <groupId>io.projectreactor</groupId>
  <artifactId>reactor-core</artifactId>
</dependency>
```

Reactor also provides some great testing support. You're going to write a lot of tests around your Reactor code, so you'll definitely want to add the next dependency to your build:

```
<dependency>
  <groupId>io.projectreactor</groupId>
  <artifactId>reactor-test</artifactId>
  <scope>test</scope>
</dependency>
```

I'm assuming that you're adding these dependencies to a Spring Boot project, which handles dependency management for you, so there's no need to specify the `<version>` element for the dependencies. But if you want to use Reactor in a non–Spring Boot project, you'll need to set up Reactor's BOM (bill of materials) in the build. The following dependency management entry adds Reactor's 2020.0.4 release to the build:

```
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>io.projectreactor</groupId>
            <artifactId>reactor-bom</artifactId>
            <version>2020.0.4</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

The examples we'll work with in this chapter are standalone and unrelated to the Taco Cloud projects we've been working with. Therefore, it may be best to create a fresh new Spring project with the Reactor dependencies in the build and work from there.

Now that Reactor is in your project build, you can start creating reactive pipelines with `Mono` and `Flux`. For the remainder of this chapter, we'll walk through several operations offered by `Mono` and `Flux`.

11.3 Applying common reactive operations

Flux and Mono are the most essential building blocks provided by Reactor, and the operations those two reactive types offer are the mortar that binds them together to create pipelines through which data can flow. Flux and Mono offer more than 500 operations, which can be loosely categorized as follows:

- Creation
- Combination
- Transformation
- Logic

As much fun as it would be to poke at each of the 500 operations to see how they tick, there's simply not enough room in this chapter. I've selected a few of the most useful operations to experiment with in this section. We'll start with creation operations.

> **NOTE** Where are the Mono examples? Mono and Flux share many of the same operations, so it's mostly unnecessary to show the same operation twice, once for Mono and again for Flux. Moreover, although the Mono operations are useful, they're slightly less interesting to look at than the same operations when given a Flux. Most of the examples we'll work with will involve Flux. Just know that Mono often has equivalent operations.

11.3.1 Creating reactive types

Often when working with reactive types in Spring, you'll be given a Flux or a Mono from a repository or a service, so you won't need to create one yourself. But occasionally you'll need to create a new reactive publisher.

Reactor provides several operations for creating a Flux or Mono. In this section, we'll look at a few of the most useful creation operations.

CREATING FROM OBJECTS

If you have one or more objects from which you'd like to create a Flux or Mono, you can use the static just() method on Flux or Mono to create a reactive type whose data is driven by those objects. For example, the following test method creates a Flux from five String objects:

```
@Test
public void createAFlux_just() {
  Flux<String> fruitFlux = Flux
      .just("Apple", "Orange", "Grape", "Banana", "Strawberry");
}
```

At this point, the Flux has been created, but it has no subscribers. Without any subscribers, data won't flow. Thinking of the garden hose analogy, you've attached the garden hose to the spigot, and there's water from the utility company on the other side—but until you turn on the spigot, water won't flow. Subscribing to a reactive type is how you turn on the flow of data.

To add a subscriber, you can call the `subscribe()` method on the `Flux` as follows:

```
fruitFlux.subscribe(
  f -> System.out.println("Here's some fruit: " + f)
);
```

The lambda given to `subscribe()` here is actually a `java.util.Consumer` that's used to create a Reactive Streams `Subscriber`. Upon calling `subscribe()`, the data starts flowing. In this example, there are no intermediate operations, so the data flows directly from the `Flux` to the `Subscriber`.

Printing the entries from a `Flux` or `Mono` to the console is a good way to see the reactive type in action. But a better way to actually test a `Flux` or a `Mono` is to use Reactor's `StepVerifier`. Given a `Flux` or `Mono`, `StepVerifier` subscribes to the reactive type and then applies assertions against the data as it flows through the stream, finally verifying that the stream completes as expected.

For example, to verify that the prescribed data flows through the `fruitFlux`, you can write a test that looks like this:

```
StepVerifier.create(fruitFlux)
    .expectNext("Apple")
    .expectNext("Orange")
    .expectNext("Grape")
    .expectNext("Banana")
    .expectNext("Strawberry")
    .verifyComplete();
```

In this case, `StepVerifier` subscribes to the `Flux` and then asserts that each item matches the expected fruit name. Finally, it verifies that after `Strawberry` is produced by the `Flux`, the `Flux` is complete.

For the remainder of the examples in this chapter, you'll use `StepVerifier` to write learning tests—tests that verify behavior and help you understand how something works—to get to know some of Reactor's most useful operations.

CREATING FROM COLLECTIONS

A `Flux` can also be created from an array, `Iterable`, or Java `Stream`. Figure 11.3 illustrates how this works with a marble diagram.

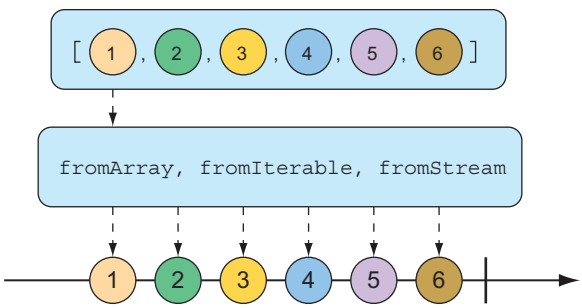

Figure 11.3 A `Flux` can be created from an array, `Iterable`, or `Stream`.

To create a `Flux` from an array, call the static `fromArray()` method, passing in the source array like so:

```
@Test
public void createAFlux_fromArray() {
  String[] fruits = new String[] {
      "Apple", "Orange", "Grape", "Banana", "Strawberry" };

  Flux<String> fruitFlux = Flux.fromArray(fruits);

  StepVerifier.create(fruitFlux)
      .expectNext("Apple")
      .expectNext("Orange")
      .expectNext("Grape")
      .expectNext("Banana")
      .expectNext("Strawberry")
      .verifyComplete();
}
```

Because the source array contains the same fruit names you used when creating a `Flux` from a list of objects, the data emitted by the `Flux` will have the same values. Thus, you can use the same `StepVerifier` as before to verify this `Flux`.

If you need to create a `Flux` from a `java.util.List`, `java.util.Set`, or any other implementation of `java.lang.Iterable`, you can pass it into the static `fromIterable()` method, as shown here:

```
@Test
public void createAFlux_fromIterable() {
  List<String> fruitList = new ArrayList<>();
  fruitList.add("Apple");
  fruitList.add("Orange");
  fruitList.add("Grape");
  fruitList.add("Banana");
  fruitList.add("Strawberry");

  Flux<String> fruitFlux = Flux.fromIterable(fruitList);

  StepVerifier.create(fruitFlux)
      .expectNext("Apple")
      .expectNext("Orange")
      .expectNext("Grape")
      .expectNext("Banana")
      .expectNext("Strawberry")
      .verifyComplete();
}
```

Or, if you happen to have a Java `Stream` that you'd like to use as the source for a `Flux`, `fromStream()` is the method you'll use, as shown next:

```
@Test
 public void createAFlux_fromStream() {
   Stream<String> fruitStream =
        Stream.of("Apple", "Orange", "Grape", "Banana", "Strawberry");
```

```
    Flux<String> fruitFlux = Flux.fromStream(fruitStream);

    StepVerifier.create(fruitFlux)
        .expectNext("Apple")
        .expectNext("Orange")
        .expectNext("Grape")
        .expectNext("Banana")
        .expectNext("Strawberry")
        .verifyComplete();
}
```

Again, you can use the same `StepVerifier` as before to verify the data published by the `Flux`.

GENERATING FLUX DATA

Sometimes you don't have any data to work with and just need `Flux` to act as a counter, emitting a number that increments with each new value. To create a counter `Flux`, you can use the static `range()` method. The diagram in figure 11.4 illustrates how `range()` works.

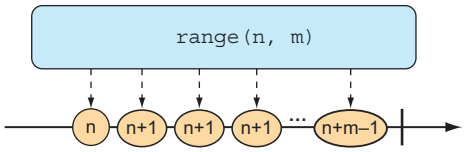

Figure 11.4 Creating a `Flux` from a range results in a counter-style publishing of messages.

The following test method demonstrates how to create a range `Flux`:

```
@Test
public void createAFlux_range() {
  Flux<Integer> intervalFlux =
      Flux.range(1, 5);

  StepVerifier.create(intervalFlux)
      .expectNext(1)
      .expectNext(2)
      .expectNext(3)
      .expectNext(4)
      .expectNext(5)
      .verifyComplete();
}
```

In this example, the range `Flux` is created with a starting value of `1` and an ending value of `5`. The `StepVerifier` proves that it will publish five items, which are the integers 1 through 5.

Another `Flux`-creation method that's similar to `range()` is `interval()`. Like the `range()` method, `interval()` creates a `Flux` that emits an incrementing value. But what makes `interval()` special is that instead of you giving it a starting and ending

value, you specify a duration or how often a value should be emitted. Figure 11.5 shows a marble diagram for the `interval()` creation method.

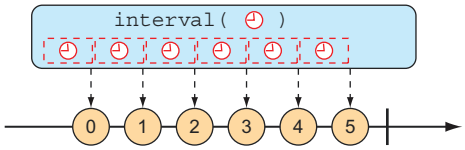

Figure 11.5 A `Flux` created from an interval has a periodic entry published to it.

For example, to create an interval `Flux` that emits a value every second, you can use the static `interval()` method as follows:

```
@Test
public void createAFlux_interval() {
  Flux<Long> intervalFlux =
      Flux.interval(Duration.ofSeconds(1))
          .take(5);

  StepVerifier.create(intervalFlux)
     .expectNext(0L)
      .expectNext(1L)
      .expectNext(2L)
      .expectNext(3L)
      .expectNext(4L)
      .verifyComplete();
}
```

Notice that the value emitted by an interval `Flux` starts with `0` and increments on each successive item. Also, because `interval()` isn't given a maximum value, it will potentially run forever. Therefore, you also use the `take()` operation to limit the results to the first five entries. We'll talk more about the `take()` operation in the next section.

11.3.2 Combining reactive types

You may find yourself with two reactive types that you need to somehow merge together. Or, in other cases, you may need to split a `Flux` into more than one reactive type. In this section, we'll examine operations that combine and split Reactor's `Flux` and `Mono`.

Merging reactive types

Suppose you have two `Flux` streams and need to create a single resulting `Flux` that will produce data as it becomes available from either of the upstream `Flux` streams. To merge one `Flux` with another, you can use the `mergeWith()` operation, as illustrated with the marble diagram in figure 11.6.

For example, suppose you have a `Flux` whose values are the names of TV and movie characters, and you have a second `Flux` whose values are the names of foods

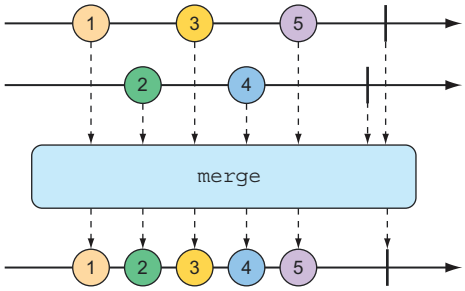

Figure 11.6 Merging two `Flux` streams interleaves their messages into a new `Flux`.

that those characters enjoy eating. The following test method shows how you could merge the two `Flux` objects with the `mergeWith()` method:

```
@Test
public void mergeFluxes() {

  Flux<String> characterFlux = Flux
      .just("Garfield", "Kojak", "Barbossa")
      .delayElements(Duration.ofMillis(500));
  Flux<String> foodFlux = Flux
      .just("Lasagna", "Lollipops", "Apples")
      .delaySubscription(Duration.ofMillis(250))
      .delayElements(Duration.ofMillis(500));

  Flux<String> mergedFlux = characterFlux.mergeWith(foodFlux);

  StepVerifier.create(mergedFlux)
      .expectNext("Garfield")
      .expectNext("Lasagna")
      .expectNext("Kojak")
      .expectNext("Lollipops")
      .expectNext("Barbossa")
      .expectNext("Apples")
      .verifyComplete();
}
```

Normally, a `Flux` will publish data as quickly as it possibly can. Therefore, you use a `delayElements()` operation on both of the created `Flux` streams to slow them down a little—emitting an entry only every 500 ms. Furthermore, so that the food `Flux` starts streaming after the character `Flux`, you apply a `delaySubscription()` operation to the food `Flux` so that it won't emit any data until 250 ms have passed following a subscription.

After merging the two `Flux` objects, a new merged `Flux` is created. When `StepVerifier` subscribes to the merged `Flux`, it will, in turn, subscribe to the two source `Flux` streams, starting the flow of data.

The order of items emitted from the merged `Flux` aligns with the timing of how they're emitted from the sources. Because both `Flux` objects are set to emit at regular

rates, the values will be interleaved through the merged `Flux`, resulting in a character, followed by a food, followed by a character, and so forth. If the timing of either `Flux` were to change, it's possible that you might see two character items or two food items published one after the other.

Because `mergeWith()` can't guarantee a perfect back and forth between its sources, you may want to consider the `zip()` operation instead. When two `Flux` objects are zipped together, it results in a new `Flux` that produces a tuple of items, where the tuple contains one item from each source `Flux`. Figure 11.7 illustrates how two `Flux` objects can be zipped together.

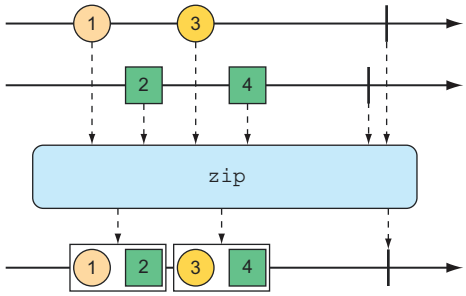

Figure 11.7 Zipping two `Flux` streams results in a `Flux` containing tuples of one element from each `Flux`.

To see the `zip()` operation in action, consider the following test method, which zips the character `Flux` and the food `Flux` together:

```
@Test
public void zipFluxes() {
  Flux<String> characterFlux = Flux
      .just("Garfield", "Kojak", "Barbossa");
  Flux<String> foodFlux = Flux
      .just("Lasagna", "Lollipops", "Apples");

  Flux<Tuple2<String, String>> zippedFlux =
      Flux.zip(characterFlux, foodFlux);

  StepVerifier.create(zippedFlux)
        .expectNextMatches(p ->
            p.getT1().equals("Garfield") &&
            p.getT2().equals("Lasagna"))
        .expectNextMatches(p ->
            p.getT1().equals("Kojak") &&
            p.getT2().equals("Lollipops"))
        .expectNextMatches(p ->
            p.getT1().equals("Barbossa") &&
            p.getT2().equals("Apples"))
        .verifyComplete();
}
```

Notice that unlike `mergeWith()`, the `zip()` operation is a static creation operation. The created `Flux` has a perfect alignment between characters and their favorite foods.

Each item emitted from the zipped `Flux` is a `Tuple2` (a container object that carries two other objects) containing items from each source `Flux`, in the order that they're published.

If you'd rather not work with a `Tuple2` and would rather work with some other type, you can provide a `Function` to `zip()` that produces any object you'd like, given the two items (as shown in the marble diagram in figure 11.8).

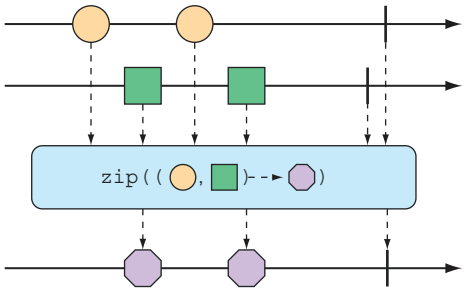

Figure 11.8 An alternative form of the `zip()` operation results in a `Flux` of messages created from one element of each incoming `Flux`.

For example, the following test method shows how to zip the character `Flux` with the food `Flux` so that it results in a `Flux` of `String` objects:

```
@Test
public void zipFluxesToObject() {
  Flux<String> characterFlux = Flux
      .just("Garfield", "Kojak", "Barbossa");
  Flux<String> foodFlux = Flux
      .just("Lasagna", "Lollipops", "Apples");

  Flux<String> zippedFlux =
      Flux.zip(characterFlux, foodFlux, (c, f) -> c + " eats " + f);

  StepVerifier.create(zippedFlux)
        .expectNext("Garfield eats Lasagna")
        .expectNext("Kojak eats Lollipops")
        .expectNext("Barbossa eats Apples")
        .verifyComplete();
}
```

The `Function` given to `zip()` (given here as a lambda) simply concatenates the two items into a sentence to be emitted by the zipped `Flux`.

SELECTING THE FIRST REACTIVE TYPE TO PUBLISH

Suppose you have two `Flux` objects, and rather than merge them together, you merely want to create a new `Flux` that emits the values from the first `Flux` that produces a value. As shown in figure 11.9, the `firstWithSignal()` operation picks the first of two `Flux` objects and echoes the values it publishes.

The following test method creates a fast `Flux` and a slow `Flux` (where "slow" means that it will not publish an item until 100 ms after subscription). Using `firstWithSignal()`,

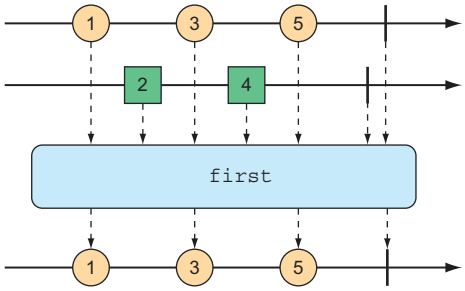

Figure 11.9 The `first()` operation chooses the first `Flux` to emit a message and thereafter produces messages only from that `Flux`.

it creates a new `Flux` that will publish values only from the first source `Flux` to publish a value.

```
@Test
public void firstWithSignalFlux() {

  Flux<String> slowFlux = Flux.just("tortoise", "snail", "sloth")
        .delaySubscription(Duration.ofMillis(100));
  Flux<String> fastFlux = Flux.just("hare", "cheetah", "squirrel");

  Flux<String> firstFlux = Flux.firstWithSignal(slowFlux, fastFlux);

  StepVerifier.create(firstFlux)
      .expectNext("hare")
      .expectNext("cheetah")
      .expectNext("squirrel")
      .verifyComplete();
}
```

In this case, because the slow `Flux` won't publish any values until 100 ms after the fast `Flux` has started publishing, the newly created `Flux` will simply ignore the slow `Flux` and publish values only from the fast `Flux`.

11.3.3 Transforming and filtering reactive streams

As data flows through a stream, you'll likely need to filter out some values and modify other values. In this section, we'll look at operations that transform and filter the data flowing through a reactive stream.

Filtering data from reactive types

One of the most basic ways of filtering data as it flows from a `Flux` is to simply disregard the first so many entries. The `skip()` operation, illustrated in figure 11.10, does exactly that.

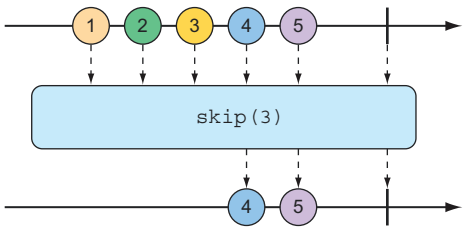

Figure 11.10 The `skip()` operation skips a specified number of messages before passing the remaining messages on to the resulting `Flux`.

Given a `Flux` with several entries, the `skip()` operation will create a new `Flux` that skips over a specified number of items before emitting the remaining items from the source `Flux`. The following test method shows how to use `skip()`:

```
@Test
public void skipAFew() {
  Flux<String> countFlux = Flux.just(
      "one", "two", "skip a few", "ninety nine", "one hundred")
      .skip(3);

  StepVerifier.create(countFlux)
      .expectNext("ninety nine", "one hundred")
      .verifyComplete();
}
```

In this case, you have a `Flux` of five `String` items. Calling `skip(3)` on that `Flux` produces a new `Flux` that skips over the first three items and publishes only the last two items.

But maybe you don't want to skip a specific number of items but instead need to skip the first so many items until some duration has passed. An alternate form of the `skip()` operation, illustrated in figure 11.11, produces a `Flux` that waits until some specified time has passed before emitting items from the source `Flux`.

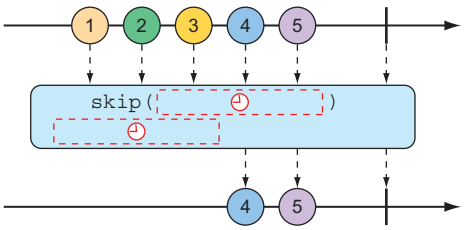

Figure 11.11 An alternative form of the `skip()` operation waits until some duration has passed before passing messages on to the resulting `Flux`.

The test method that follows uses `skip()` to create a `Flux` that waits 4 seconds before emitting any values. Because that `Flux` was created from a `Flux` that has a 1-second delay between items (using `delayElements()`), only the last two items will be emitted.

```
@Test
public void skipAFewSeconds() {
  Flux<String> countFlux = Flux.just(
      "one", "two", "skip a few", "ninety nine", "one hundred")
      .delayElements(Duration.ofSeconds(1))
      .skip(Duration.ofSeconds(4));

  StepVerifier.create(countFlux)
      .expectNext("ninety nine", "one hundred")
      .verifyComplete();
}
```

You've already seen an example of the `take()` operation, but in light of the `skip()` operation, `take()` can be thought of as the opposite of `skip()`. Whereas `skip()` skips the first few items, `take()` emits only the first so many items (as illustrated by the marble diagram in figure 11.12):

```
@Test
public void take() {
  Flux<String> nationalParkFlux = Flux.just(
      "Yellowstone", "Yosemite", "Grand Canyon", "Zion", "Acadia")
      .take(3);

  StepVerifier.create(nationalParkFlux)
      .expectNext("Yellowstone", "Yosemite", "Grand Canyon")
      .verifyComplete();
}
```

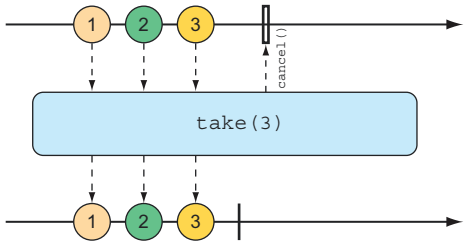

Figure 11.12 The `take()` operation passes only the first so many messages from the incoming `Flux` and then cancels the subscription.

Like `skip()`, `take()` also has an alternative form that's based on a duration rather than an item count. It will take and emit as many items as pass through the source `Flux` until some period of time has passed, after which the `Flux` completes. This is illustrated in figure 11.13.

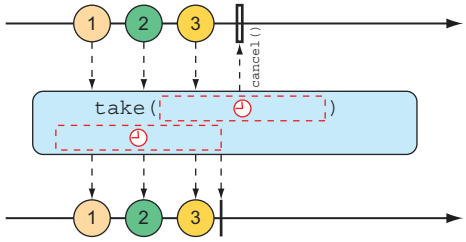

Figure 11.13 An alternative form of the `take()` operation passes messages on to the resulting `Flux` until some duration has passed.

The following test method uses the alternative form of `take()` to emit as many items as it can in the first 3.5 seconds after subscription:

```
@Test
public void takeForAwhile() {
```

```
  Flux<String> nationalParkFlux = Flux.just(
      "Yellowstone", "Yosemite", "Grand Canyon", "Zion", "Grand Teton")
      .delayElements(Duration.ofSeconds(1))
      .take(Duration.ofMillis(3500));

  StepVerifier.create(nationalParkFlux)
      .expectNext("Yellowstone", "Yosemite", "Grand Canyon")
      .verifyComplete();
}
```

The `skip()` and `take()` operations can be thought of as filter operations where the filter criteria are based on a count or a duration. For more general-purpose filtering of `Flux` values, you'll find the `filter()` operation quite useful.

Given a `Predicate` that decides whether an item will pass through the `Flux`, the `filter()` operation lets you selectively publish based on whatever criteria you want. The marble diagram in figure 11.14 shows how `filter()` works.

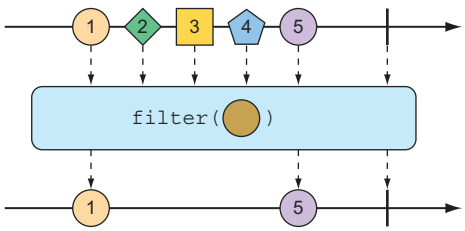

Figure 11.14 An incoming `Flux` can be filtered so that the resulting `Flux` receives only messages that match a given predicate.

To see `filter()` in action, consider the following test method:

```
@Test+
public void filter() {
  Flux<String> nationalParkFlux = Flux.just(
      "Yellowstone", "Yosemite", "Grand Canyon", "Zion", "Grand Teton")
      .filter(np -> !np.contains(" "));

  StepVerifier.create(nationalParkFlux)
      .expectNext("Yellowstone", "Yosemite", "Zion")
      .verifyComplete();
}
```

Here, `filter()` is given a `Predicate` as a lambda that accepts only `String` values that don't have any spaces. Consequently, `"Grand Canyon"` and `"Grand Teton"` are filtered out of the resulting `Flux`.

Perhaps the filtering you need is to filter out any items that you've already received. The `distinct()` operation, as illustrated in figure 11.15, results in a `Flux` that publishes only items from the source `Flux` that haven't already been published.

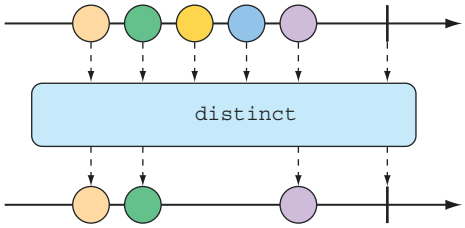

Figure 11.15 The `distinct()` operation filters out any duplicate messages.

In the following test, only unique `String` values will be emitted from the distinct `Flux`:

```
@Test
public void distinct() {
  Flux<String> animalFlux = Flux.just(
      "dog", "cat", "bird", "dog", "bird", "anteater")
      .distinct();

  StepVerifier.create(animalFlux)
      .expectNext("dog", "cat", "bird", "anteater")
      .verifyComplete();
}
```

Although `"dog"` and `"bird"` are each published twice from the source `Flux`, the distinct `Flux` publishes them only once.

Mapping reactive data

One of the most common operations you'll use on either a `Flux` or a `Mono` is to transform published items to some other form or type. Reactor's types offer `map()` and `flatMap()` operations for that purpose.

The `map()` operation creates a `Flux` that simply performs a transformation as prescribed by a given `Function` on each object it receives before republishing it. Figure 11.16 illustrates how the `map()` operation works.

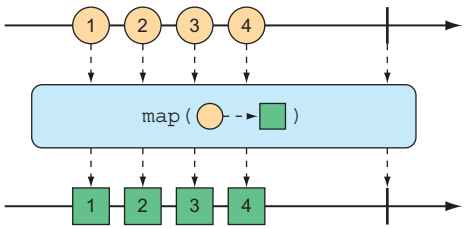

Figure 11.16 The `map()` operation performs a transformation of incoming messages into new messages on the resulting stream.

In the following test method, a `Flux` of `String` values representing basketball players is mapped to a new `Flux` of `Player` objects:

```
@Test
public void map() {
```

```
  Flux<Player> playerFlux = Flux
    .just("Michael Jordan", "Scottie Pippen", "Steve Kerr")
    .map(n -> {
      String[] split = n.split("\\s");
      return new Player(split[0], split[1]);
    });

  StepVerifier.create(playerFlux)
      .expectNext(new Player("Michael", "Jordan"))
      .expectNext(new Player("Scottie", "Pippen"))
      .expectNext(new Player("Steve", "Kerr"))
      .verifyComplete();
}

@Data
private static class Player {
  private final String firstName;
  private final String lastName;
}
```

The `Function` given to `map()` (as a lambda) splits the incoming `String` at a space and uses the resulting `String` array to create a `Player` object. Although the `Flux` created with `just()` carried `String` objects, the `Flux` resulting from `map()` carries `Player` objects.

What's important to understand about `map()` is that the mapping is performed synchronously, as each item is published by the source `Flux`. If you want to perform the mapping asynchronously, you should consider the `flatMap()` operation.

The `flatMap()` operation requires some thought and practice to acquire full proficiency. As shown in figure 11.17, instead of simply mapping one object to another, as in the case of `map()`, `flatMap()` maps each object to a new `Mono` or `Flux`. The results of the `Mono` or `Flux` are flattened into a new resulting `Flux`. When used along with `subscribeOn()`, `flatMap()` can unleash the asynchronous power of Reactor's types.

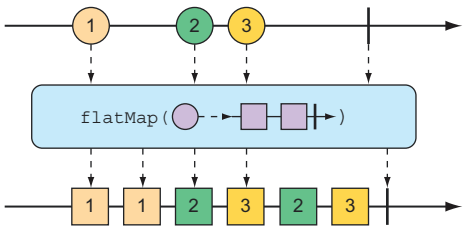

Figure 11.17 The `flatMap()` operation uses an intermediate `Flux` to perform a transformation, consequently allowing for asynchronous transformations.

The following test method demonstrates the use of `flatMap()` and `subscribeOn()`:

```
@Test
public void flatMap() {
  Flux<Player> playerFlux = Flux
    .just("Michael Jordan", "Scottie Pippen", "Steve Kerr")
```

```
    .flatMap(n -> Mono.just(n)
        .map(p -> {
            String[] split = p.split("\\s");
            return new Player(split[0], split[1]);
          })
        .subscribeOn(Schedulers.parallel())
      );

  List<Player> playerList = Arrays.asList(
      new Player("Michael", "Jordan"),
      new Player("Scottie", "Pippen"),
      new Player("Steve", "Kerr"));

  StepVerifier.create(playerFlux)
      .expectNextMatches(p -> playerList.contains(p))
      .expectNextMatches(p -> playerList.contains(p))
      .expectNextMatches(p -> playerList.contains(p))
      .verifyComplete();
}
```

Notice that `flatMap()` is given a lambda `Function` that transforms the incoming `String` into a `Mono` of type `String`. A `map()` operation is then applied to the `Mono` to transform the `String` into a `Player`. After the `String` is mapped to a `Player` on each internal `Flux`, they are published into a single `Flux` returned by `flatMap()`, thus completing the flattening of the results.

If you stopped right there, the resulting `Flux` would carry `Player` objects, produced synchronously in the same order as with the `map()` example. But the last thing you do with the `Mono` is call `subscribeOn()` to indicate that each subscription should take place in a parallel thread. Consequently, the mapping operations for multiple incoming `String` objects can be performed asynchronously and in parallel.

Although `subscribeOn()` is named similarly to `subscribe()`, they're quite different. Whereas `subscribe()` is a verb, subscribing to a reactive flow and effectively kicking it off, `subscribeOn()` is more descriptive, specifying *how* a subscription should be handled concurrently. Reactor doesn't force any particular concurrency model; it's through `subscribeOn()` that you can specify the concurrency model, using one of the static methods from `Schedulers`, that you want to use. In this example, you used `parallel()`, which uses worker threads from a fixed pool (sized to the number of CPU cores). But `Schedulers` supports several concurrency models, such as those described in table 11.1.

Table 11.1 Concurrency models for `Schedulers`

`Schedulers` method	Description
`.immediate()`	Executes the subscription in the current thread.
`.single()`	Executes the subscription in a single, reusable thread. Reuses the same thread for all callers.
`.newSingle()`	Executes the subscription in a per-call dedicated thread.

Table 11.1 Concurrency models for `Schedulers` *(continued)*

`Schedulers` method	Description
`.elastic()`	Executes the subscription in a worker pulled from an unbounded, elastic pool. New worker threads are created as needed, and idle workers are disposed of (by default, after 60 seconds).
`.parallel()`	Executes the subscription in a worker pulled from a fixed-size pool, sized to the number of CPU cores.

The upside to using `flatMap()` and `subscribeOn()` is that you can increase the throughput of the stream by splitting the work across multiple parallel threads. But because the work is being done in parallel, with no guarantees on which will finish first, there's no way to know the order of items emitted in the resulting `Flux`. Therefore, `StepVerifier` is able to verify only that each item emitted exists in the expected list of `Player` objects and that there will be three such items before the `Flux` completes.

BUFFERING DATA ON A REACTIVE STREAM

In the course of processing the data flowing through a `Flux`, you might find it helpful to break the stream of data into bite-size chunks. The `buffer()` operation, shown in figure 11.18, can help with that.

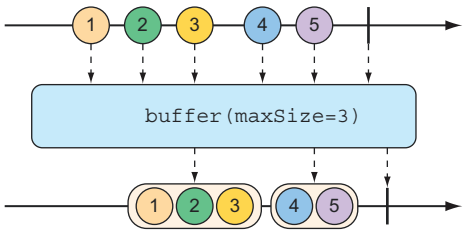

Figure 11.18 The `buffer()` operation results in a `Flux` of lists of a given maximum size that are collected from the incoming `Flux`.

Given a `Flux` of `String` values, each containing the name of a fruit, you can create a new `Flux` of `List` collections, where each `List` has no more than a specified number of elements as follows:

```
@Test
public void buffer() {
  Flux<String> fruitFlux = Flux.just(
      "apple", "orange", "banana", "kiwi", "strawberry");

  Flux<List<String>> bufferedFlux = fruitFlux.buffer(3);

  StepVerifier
      .create(bufferedFlux)
      .expectNext(Arrays.asList("apple", "orange", "banana"))
      .expectNext(Arrays.asList("kiwi", "strawberry"))
      .verifyComplete();
}
```

In this case, the `Flux` of `String` elements is buffered into a new `Flux` of `List` collections containing no more than three items each. Consequently, the original `Flux` that emits five `String` values will be converted to a `Flux` that emits two `List` collections, one containing three fruits and the other with two fruits.

So what? Buffering values from a reactive `Flux` into nonreactive `List` collections seems counterproductive. But when you combine `buffer()` with `flatMap()`, it enables each of the `List` collections to be processed in parallel, as shown next:

```
@Test
public void bufferAndFlatMap() throws Exception {
  Flux.just(
      "apple", "orange", "banana", "kiwi", "strawberry")
      .buffer(3)
      .flatMap(x ->
        Flux.fromIterable(x)
          .map(y -> y.toUpperCase())
          .subscribeOn(Schedulers.parallel())
          .log()
      ).subscribe();
}
```

In this new example, you still buffer a `Flux` of five `String` values into a new `Flux` of `List` collections. But then you apply `flatMap()` to that `Flux` of `List` collections. This takes each `List` buffer and creates a new `Flux` from its elements, and then applies a `map()` operation on it. Consequently, each buffered `List` is further processed in parallel in individual threads.

To prove that it works, I've also included a `log()` operation to be applied to each sub-`Flux`. The `log()` operation simply logs all Reactive Streams events, so that you can see what's really happening. As a result, the following entries are written to the log (with the time component removed for brevity's sake):

```
[main] INFO reactor.Flux.SubscribeOn.1 -
                  onSubscribe(FluxSubscribeOn.SubscribeOnSubscriber)
[main] INFO reactor.Flux.SubscribeOn.1 - request(32)
[main] INFO reactor.Flux.SubscribeOn.2 -
                  onSubscribe(FluxSubscribeOn.SubscribeOnSubscriber)
[main] INFO reactor.Flux.SubscribeOn.2 - request(32)
[parallel-1] INFO reactor.Flux.SubscribeOn.1 - onNext(APPLE)
[parallel-2] INFO reactor.Flux.SubscribeOn.2 - onNext(KIWI)
[parallel-1] INFO reactor.Flux.SubscribeOn.1 - onNext(ORANGE)
[parallel-2] INFO reactor.Flux.SubscribeOn.2 - onNext(STRAWBERRY)
[parallel-1] INFO reactor.Flux.SubscribeOn.1 - onNext(BANANA)
[parallel-1] INFO reactor.Flux.SubscribeOn.1 - onComplete()
[parallel-2] INFO reactor.Flux.SubscribeOn.2 - onComplete()
```

As the log entries clearly show, the fruits in the first buffer (apple, orange, and banana) are handled in the `parallel-1` thread. Meanwhile, the fruits in the second buffer (kiwi and strawberry) are processed in the `parallel-2` thread. As is apparent

by the fact that the log entries from each buffer are woven together, the two buffers are processed in parallel.

If, for some reason, you need to collect everything that a `Flux` emits into a `List`, you can call `buffer()` with no arguments as follows:

```
Flux<List<String>> bufferedFlux = fruitFlux.buffer();
```

This results in a new `Flux` that emits a `List` that contains all the items published by the source `Flux`. You can achieve the same thing with the `collectList()` operation, illustrated by the marble diagram in figure 11.19.

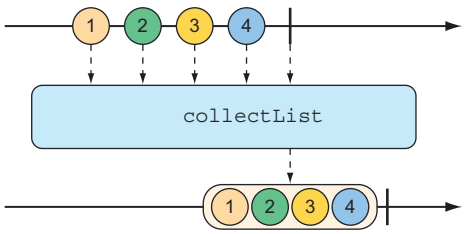

Figure 11.19 The `collectList()` operation results in a `Mono` containing a list of all messages emitted by the incoming `Flux`.

Rather than produce a `Flux` that publishes a `List`, `collectList()` produces a `Mono` that publishes a `List`. The following test method shows how it might be used:

```
@Test
public void collectList() {
  Flux<String> fruitFlux = Flux.just(
      "apple", "orange", "banana", "kiwi", "strawberry");

  Mono<List<String>> fruitListMono = fruitFlux.collectList();

  StepVerifier
      .create(fruitListMono)
      .expectNext(Arrays.asList(
          "apple", "orange", "banana", "kiwi", "strawberry"))
      .verifyComplete();
}
```

An even more interesting way of collecting items emitted by a `Flux` is to collect them into a `Map`. As shown in figure 11.20, the `collectMap()` operation results in a `Mono`

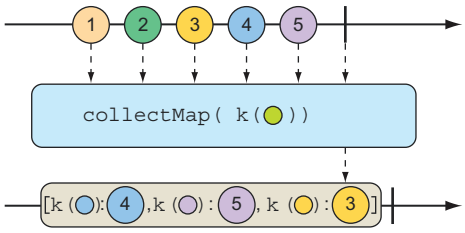

Figure 11.20 The `collectMap()` operation results in a `Mono` containing a map of messages emitted by the incoming `Flux`, where the key is derived from some characteristic of the incoming messages.

that publishes a Map that's populated with entries whose key is calculated by a given Function.

To see collectMap() in action, have a look at the following test method:

```
@Test
public void collectMap() {
  Flux<String> animalFlux = Flux.just(
      "aardvark", "elephant", "koala", "eagle", "kangaroo");

  Mono<Map<Character, String>> animalMapMono =
      animalFlux.collectMap(a -> a.charAt(0));

  StepVerifier
      .create(animalMapMono)
      .expectNextMatches(map -> {
        return
            map.size() == 3 &&
            map.get('a').equals("aardvark") &&
            map.get('e').equals("eagle") &&
            map.get('k').equals("kangaroo");
      })
      .verifyComplete();
}
```

The source Flux emits the names of a handful of animals. From that Flux, you use collectMap() to create a new Mono that emits a Map, where the key value is determined by the first letter of the animal name and the value is the animal name itself. In the event that two animal names start with the same letter (as with *elephant* and *eagle* or *koala* and *kangaroo*), the last entry flowing through the stream overrides any earlier entries.

11.3.4 Performing logic operations on reactive types

Sometimes you just need to know if the entries published by a Mono or Flux match some criteria. The all() and any() operations perform such logic. Figures 11.21 and 11.22 illustrate how all() and any() work.

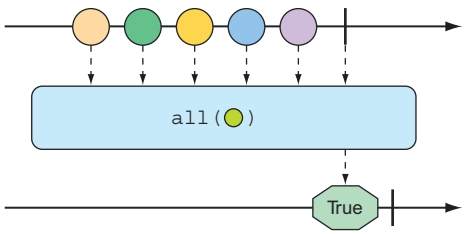

Figure 11.21 A Flux can be tested to ensure that all messages meet some condition with the all() operation.

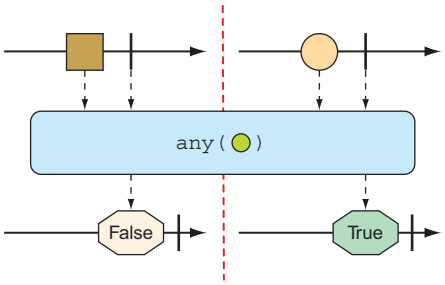

Figure 11.22 A `Flux` can be tested to ensure that at least one message meets some condition with the `any()` operation.

Suppose you want to know that every `String` published by a `Flux` contains the letter *a* or the letter *k*. The following test shows how to use `all()` to check for that condition:

```
@Test
public void all() {
  Flux<String> animalFlux = Flux.just(
      "aardvark", "elephant", "koala", "eagle", "kangaroo");

  Mono<Boolean> hasAMono = animalFlux.all(a -> a.contains("a"));
  StepVerifier.create(hasAMono)
    .expectNext(true)
    .verifyComplete();

  Mono<Boolean> hasKMono = animalFlux.all(a -> a.contains("k"));
  StepVerifier.create(hasKMono)
    .expectNext(false)
    .verifyComplete();
}
```

In the first `StepVerifier`, you check for the letter *a*. The `all` operation is applied to the source `Flux`, resulting in a `Mono` of type `Boolean`. In this case, all of the animal names contain the letter *a*, so `true` is emitted from the resulting `Mono`. But in the second `StepVerifier`, the resulting `Mono` will emit `false` because not all of the animal names contain a *k*.

Rather than perform an all-or-nothing check, maybe you're satisfied if at least one entry matches. In that case, the `any()` operation is what you want. This new test case uses `any()` to check for the letters *t* and *z*:

```
@Test
public void any() {
  Flux<String> animalFlux = Flux.just(
      "aardvark", "elephant", "koala", "eagle", "kangaroo");

  Mono<Boolean> hasTMono = animalFlux.any(a -> a.contains("t"));

  StepVerifier.create(hasTMono)
    .expectNext(true)
    .verifyComplete();
```

```
  Mono<Boolean> hasZMono = animalFlux.any(a -> a.contains("z"));
  StepVerifier.create(hasZMono)
    .expectNext(false)
    .verifyComplete();
}
```

In the first `StepVerifier`, you see that the resulting `Mono` emits `true`, because at least one animal name has the letter *t* (specifically, *elephant*). In the second case, the resulting `Mono` emits `false`, because none of the animal names contain *z*.

Summary

- Reactive programming involves creating pipelines through which data flows.
- The Reactive Streams specification defines four types: `Publisher`, `Subscriber`, `Subscription`, and `Transformer` (which is a combination of `Publisher` and `Subscriber`).
- Project Reactor implements Reactive Streams and abstracts stream definitions into two primary types, `Flux` and `Mono`, each of which offers several hundred operations.
- Spring leverages Reactor to create reactive controllers, repositories, REST clients, and other reactive framework support.

12 Developing reactive APIs

This chapter covers

- Using Spring WebFlux
- Writing and testing reactive controllers and clients
- Consuming REST APIs
- Securing reactive web applications

Now that you've had a good introduction to reactive programming and Project Reactor, you're ready to start applying those techniques in your Spring applications. In this chapter, we're going to revisit some of the controllers you wrote in chapter 7 to take advantage of Spring's reactive programming model.

More specifically, we're going to take a look at Spring's reactive web framework—Spring WebFlux. As you'll quickly discover, Spring WebFlux is remarkably similar to Spring MVC, making it easy to apply, along with what you already know about building REST APIs in Spring.

12.1 Working with Spring WebFlux

Typical servlet web frameworks, such as Spring MVC, are blocking and multithreaded in nature, using a single thread per connection. As requests are handled, a worker thread is pulled from a thread pool to process the request. Meanwhile, the request thread is blocked until it's notified by the worker thread that it's finished.

Consequently, blocking web frameworks won't scale effectively under heavy request volume. Latency in slow worker threads makes things even worse because it'll take longer for the worker thread to be returned to the pool, ready to handle another request. In some use cases, this arrangement is perfectly acceptable. In fact, this is largely how most web applications have been developed for well over a decade. But times are changing.

The clients of those web applications have grown from people occasionally viewing websites to people frequently consuming content and using applications that coordinate with HTTP APIs. And these days, the so-called *Internet of Things* (where humans aren't even involved) yields cars, jet engines, and other nontraditional clients constantly exchanging data with web APIs. With an increasing number of clients consuming web applications, scalability is more important than ever.

Asynchronous web frameworks, in contrast, achieve higher scalability with fewer threads—generally one per CPU core. By applying a technique known as *event looping* (as illustrated in figure 12.1), these frameworks are able to handle many requests per thread, making the per-connection cost more economical.

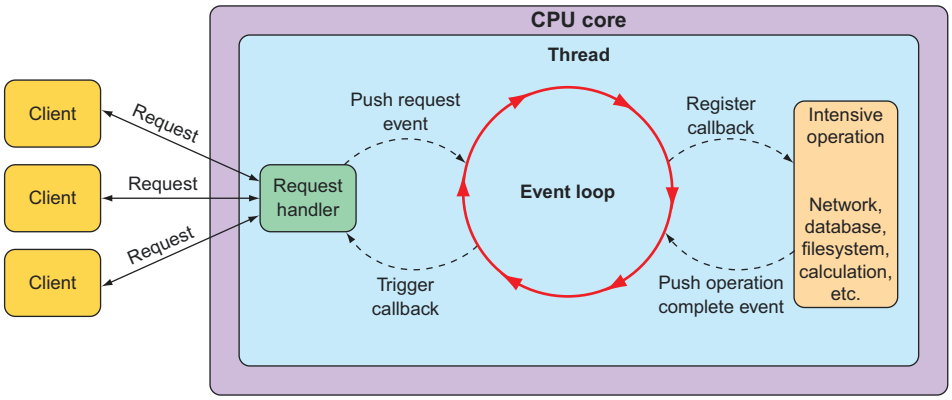

Figure 12.1 Asynchronous web frameworks apply event looping to handle more requests with fewer threads.

In an event loop, everything is handled as an event, including requests and callbacks from intensive operations like database and network operations. When a costly operation is needed, the event loop registers a callback for that operation to be performed in parallel, while it moves on to handle other events.

When the operation is complete, it's treated as an event by the event loop, the same as requests. As a result, asynchronous web frameworks are able to scale better under heavy request volume with fewer threads, resulting in reduced overhead for thread management.

Spring offers a nonblocking, asynchronous web framework based largely on its Project Reactor to address the need for greater scalability in web applications and APIs. Let's take a look at Spring WebFlux—a reactive web framework for Spring.

12.1.1 Introducing Spring WebFlux

As the Spring team was considering how to add a reactive programming model to the web layer, it quickly became apparent that it would be difficult to do so without a great deal of work in Spring MVC. That would involve branching code to decide whether or not to handle requests reactively. In essence, the result would be two web frameworks packaged as one, with `if` statements to separate the reactive from the nonreactive.

Instead of trying to shoehorn a reactive programming model into Spring MVC, the Spring team decided to create a separate reactive web framework, borrowing as much from Spring MVC as possible. Spring WebFlux is the result. Figure 12.2 illustrates the complete web development stack available in Spring.

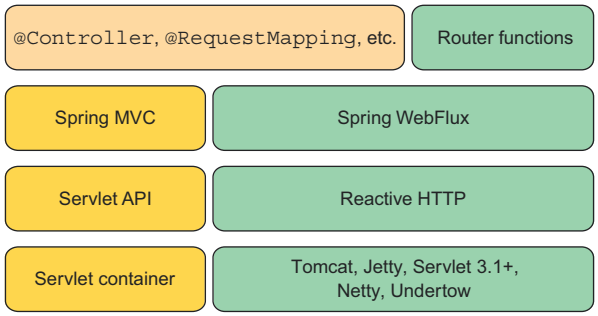

Figure 12.2 Spring supports reactive web applications with a new web framework called WebFlux, which is a sibling to Spring MVC and shares many of its core components.

On the left side of figure 12.2, you see the Spring MVC stack that was introduced in version 2.5 of the Spring Framework. Spring MVC (covered in chapters 2 and 7) sits atop the Java Servlet API, which requires a servlet container (such as Tomcat) to execute on.

By contrast, Spring WebFlux (on the right side) doesn't have ties to the Servlet API, so it builds on top of a Reactive HTTP API, which is a reactive approximation of the same functionality provided by the Servlet API. And because Spring WebFlux isn't coupled to the Servlet API, it doesn't require a servlet container to run on. Instead, it

can run on any nonblocking web container including Netty, Undertow, Tomcat, Jetty, or any Servlet 3.1 or higher container.

What's most noteworthy about figure 12.2 is the top-left box, which represents the components that are common between Spring MVC and Spring WebFlux, primarily the annotations used to define controllers. Because Spring MVC and Spring WebFlux share the same annotations, Spring WebFlux is, in many ways, indistinguishable from Spring MVC.

The box in the top-right corner represents an alternative programming model that defines controllers with a functional programming paradigm instead of using annotations. We'll talk more about Spring's functional web programming model in section 12.2.

The most significant difference between Spring MVC and Spring WebFlux boils down to which dependency you add to your build. When working with Spring WebFlux, you'll need to add the Spring Boot WebFlux starter dependency instead of the standard web starter (e.g., `spring-boot-starter-web`). In the project's pom.xml file, it looks like this:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

> **NOTE** As with most of Spring Boot's starter dependencies, this starter can also be added to a project by checking the Reactive Web check box in the Initializr.

An interesting side effect of using WebFlux instead of Spring MVC is that the default embedded server for WebFlux is Netty instead of Tomcat. Netty is one of a handful of asynchronous, event-driven servers and is a natural fit for a reactive web framework like Spring WebFlux.

Aside from using a different starter dependency, Spring WebFlux controller methods usually accept and return reactive types, like `Mono` and `Flux`, instead of domain types and collections. Spring WebFlux controllers can also deal with RxJava types like `Observable`, `Single`, and `Completable`.

Reactive Spring MVC?

Although Spring WebFlux controllers typically return `Mono` and `Flux`, that doesn't mean that Spring MVC doesn't get to have some fun with reactive types. Spring MVC controller methods can also return a `Mono` or `Flux`, if you'd like.

The difference is in how those types are used. Whereas Spring WebFlux is a truly reactive web framework, allowing for requests to be handled in an event loop, Spring MVC is servlet-based, relying on multithreading to handle multiple requests.

Let's put Spring WebFlux to work by rewriting some of Taco Cloud's API controllers to take advantage of Spring WebFlux.

12.1.2 Writing reactive controllers

You may recall that in chapter 7, you created a few controllers for Taco Cloud's REST API. Those controllers had request-handling methods that dealt with input and output in terms of domain types (such as `TacoOrder` and `Taco`) or collections of those domain types. As a reminder, consider the following snippet from `TacoController` that you wrote back in chapter 7:

```
@RestController
@RequestMapping(path="/api/tacos",
                produces="application/json")
@CrossOrigin(origins="*")
public class TacoController {

...

  @GetMapping(params="recent")
  public Iterable<Taco> recentTacos() {
    PageRequest page = PageRequest.of(
            0, 12, Sort.by("createdAt").descending());
    return tacoRepo.findAll(page).getContent();
  }

...

}
```

As written, the `recentTacos()` controller handles HTTP `GET` requests for /api/tacos?recent to return a list of recently created tacos. More specifically, it returns an `Iterable` of type `Taco`. That's primarily because that's what's returned from the repository's `findAll()` method, or, more accurately, from the `getContent()` method on the `Page` object returned from `findAll()`.

That works fine, but `Iterable` isn't a reactive type. You won't be able to apply any reactive operations on it, nor can you let the framework take advantage of it as a reactive type to split any work over multiple threads. What you'd like is for `recentTacos()` to return a `Flux<Taco>`.

A simple but somewhat limited option here is to rewrite `recentTacos()` to convert the `Iterable` to a `Flux`. And, while you're at it, you can do away with the paging code and replace it with a call to `take()` on the `Flux` as follows:

```
@GetMapping(params="recent")
public Flux<Taco> recentTacos() {
  return Flux.fromIterable(tacoRepo.findAll()).take(12);
}
```

Using `Flux.fromIterable()`, you convert the `Iterable<Taco>` to a `Flux<Taco>`. And now that you're working with a `Flux`, you can use the `take()` operation to limit the returned `Flux` to 12 `Taco` objects at most. Not only is the code simpler, it also deals with a reactive `Flux` rather than a plain `Iterable`.

Writing reactive code has been a winning move so far. But it would be even better if the repository gave you a `Flux` to start with so that you wouldn't need to do the conversion. If that were the case, then `recentTacos()` could be written to look like this:

```
@GetMapping(params="recent")
public Flux<Taco> recentTacos() {
  return tacoRepo.findAll().take(12);
}
```

That's even better! Ideally, a reactive controller will be the tip of a stack that's reactive end to end, including controllers, repositories, the database, and any services that may sit in between. Such an end-to-end reactive stack is illustrated in figure 12.3.

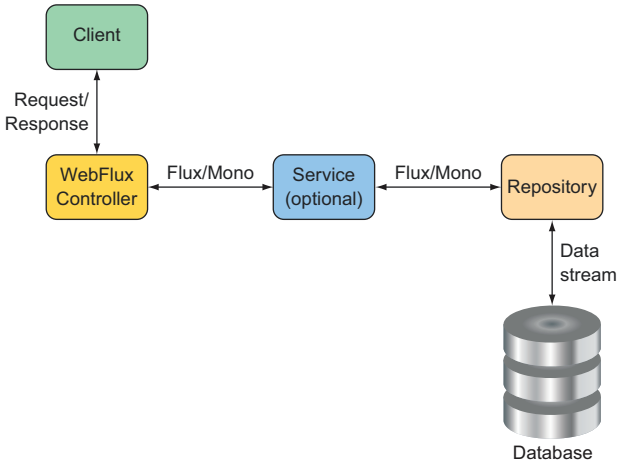

Figure 12.3 To maximize the benefit of a reactive web framework, it should be part of a full end-to-end reactive stack.

Such an end-to-end stack requires that the repository be written to return a `Flux` instead of an `Iterable`. We'll look into writing reactive repositories in the next chapter, but here's a sneak peek at what a reactive `TacoRepository` might look like:

```
package tacos.data;

import org.springframework.data.repository.reactive.ReactiveCrudRepository;
import tacos.Taco;

public interface TacoRepository
         extends ReactiveCrudRepository<Taco, Long> {
}
```

What's most important to note at this point, however, is that aside from working with a `Flux` instead of an `Iterable`, as well as how you obtain that `Flux`, the programming

model for defining a reactive WebFlux controller is no different than for a nonreactive Spring MVC controller. Both are annotated with `@RestController` and a high-level `@RequestMapping` at the class level. And both have request-handling functions that are annotated with `@GetMapping` at the method level. It's truly a matter of what type the handler methods return.

Another important observation to make is that although you're getting a `Flux<Taco>` back from the repository, you can return it without calling `subscribe()`. Indeed, the framework will call `subscribe()` for you. This means that when a request for /api/tacos?recent is handled, the `recentTacos()` method will be called and will return before the data is even fetched from the database!

RETURNING SINGLE VALUES

As another example, consider the following `tacoById()` method from the `TacoController` as it was written in chapter 7:

```
@GetMapping("/{id}")
public Taco tacoById(@PathVariable("id") Long id) {
  Optional<Taco> optTaco = tacoRepo.findById(id);
  if (optTaco.isPresent()) {
    return optTaco.get();
  }
  return null;
}
```

Here, this method handles `GET` requests for /tacos/{id} and returns a single `Taco` object. Because the repository's `findById()` returns an `Optional`, you also had to write some clunky code to deal with that. But suppose for a minute that the `findById()` returns a `Mono<Taco>` instead of an `Optional<Taco>`. In that case, you can rewrite the controller's `tacoById()` to look like this:

```
@GetMapping("/{id}")
public Mono<Taco> tacoById(@PathVariable("id") Long id) {
  return tacoRepo.findById(id);
}
```

Wow! That's a lot simpler. What's more important, however, is that by returning a `Mono<Taco>` instead of a `Taco`, you're enabling Spring WebFlux to handle the response in a reactive manner. Consequently, your API will scale better in response to heavy loads.

WORKING WITH RXJAVA TYPES

It's worth pointing out that although Reactor types like `Flux` and `Mono` are a natural choice when working with Spring WebFlux, you can also choose to work with RxJava types like `Observable` and `Single`. For example, suppose there's a service sitting between `TacoController` and the backend repository that deals in terms of RxJava types. In that case, you might write the `recentTacos()` method like this:

```
@GetMapping(params = "recent")
public Observable<Taco> recentTacos() {
  return tacoService.getRecentTacos();
}
```

Similarly, the `tacoById()` method could be written to deal with an RxJava `Single` rather than a `Mono`, as shown next:

```
@GetMapping("/{id}")
public Single<Taco> tacoById(@PathVariable("id") Long id) {
  return tacoService.lookupTaco(id);
}
```

In addition, Spring WebFlux controller methods can also return RxJava's `Completable`, which is equivalent to a `Mono<Void>` in Reactor. WebFlux can also return RxJava's `Flowable` as an alternative to `Observable` or Reactor's `Flux`.

HANDLING INPUT REACTIVELY

So far, we've concerned ourselves only with what reactive types the controller methods return. But with Spring WebFlux, you can also accept a `Mono` or a `Flux` as an input to a handler method. To demonstrate, consider the original implementation of `postTaco()` from `TacoController`, shown here:

```
@PostMapping(consumes="application/json")
@ResponseStatus(HttpStatus.CREATED)
public Taco postTaco(@RequestBody Taco taco) {
  return tacoRepo.save(taco);
}
```

As originally written, `postTaco()` not only returns a simple `Taco` object but also accepts a `Taco` object that's bound to the content in the body of the request. This means that `postTaco()` can't be invoked until the request payload has been fully resolved and used to instantiate a `Taco` object. It also means `postTaco()` can't return until the blocking call to the repository's `save()` method returns. In short, the request is blocked twice: as it enters `postTaco()` and again, inside of `postTaco()`. But by applying a little reactive coding to `postTaco()`, shown next, you can make it a fully nonblocking, request-handling method:

```
@PostMapping(consumes = "application/json")
@ResponseStatus(HttpStatus.CREATED)
public Mono<Taco> postTaco(@RequestBody Mono<Taco> tacoMono) {
  return tacoRepo.saveAll(tacoMono).next();
}
```

Here, `postTaco()` accepts a `Mono<Taco>` and calls the repository's `saveAll()` method, which accepts any implementation of Reactive Streams `Publisher`, including `Mono` or `Flux`. The `saveAll()` method returns a `Flux<Taco>`, but because you started with a

Mono, you know there's at most one Taco that will be published by the Flux. You can therefore call next() to obtain a Mono<Taco> that will return from postTaco().

By accepting a Mono<Taco> as input, the method is invoked immediately without waiting for the Taco to be resolved from the request body. And because the repository is also reactive, it'll accept a Mono and immediately return a Flux<Taco>, from which you call next() and return the resulting Mono<Taco> . . . all before the request is even processed!

Alternatively, you could also implement postTaco() like this:

```
@PostMapping(consumes = "application/json")
@ResponseStatus(HttpStatus.CREATED)
public Mono<Taco> postTaco(@RequestBody Mono<Taco> tacoMono) {
  return tacoMono.flatMap(tacoRepo::save);
}
```

This approach flips things around so that the tacoMono is the driver of the action. The Taco contained within tacoMono is handed to the repository's save() method via flatMap(), resulting in a new Mono<Taco> that is returned.

Either way works well, and there are probably several other ways that you could write postTaco(). Choose whichever way works best and makes the most sense to you.

Spring WebFlux is a fantastic alternative to Spring MVC, offering the option of writing reactive web applications using the same development model as Spring MVC. But Spring has another new trick up its sleeve. Let's take a look at how to create reactive APIs using Spring's functional programming style.

12.2 Defining functional request handlers

Spring MVC's annotation-based programming model has been around since Spring 2.5 and is widely popular. It comes with a few downsides, however.

First, any annotation-based programming involves a split in the definition of *what* the annotation is supposed to do and *how* it's supposed to do it. Annotations themselves define the what; the how is defined elsewhere in the framework code. This division complicates the programming model when it comes to any sort of customization or extension because such changes require working in code that's external to the annotation. Moreover, debugging such code is tricky because you can't set a breakpoint on an annotation.

Also, as Spring continues to grow in popularity, developers new to Spring from other languages and frameworks may find annotation-based Spring MVC (and WebFlux) quite unlike what they already know. As an alternative to WebFlux, Spring offers a functional programming model for defining reactive APIs.

This new programming model is used more like a library and less like a framework, letting you map requests to handler code without annotations. Writing an API using Spring's functional programming model involves the following four primary types:

- *RequestPredicate*—Declares the kind(s) of requests that will be handled
- *RouterFunction*—Declares how a matching request should be routed to the handler code
- *ServerRequest*—Represents an HTTP request, including access to header and body information
- *ServerResponse*—Represents an HTTP response, including header and body information

As a simple example that pulls all of these types together, consider the following Hello World example:

```
package hello;

import static org.springframework.web
                .reactive.function.server.RequestPredicates.GET;
import static org.springframework.web
                .reactive.function.server.RouterFunctions.route;
import static org.springframework.web
                .reactive.function.server.ServerResponse.ok;
import static reactor.core.publisher.Mono.just;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.reactive.function.server.RouterFunction;

@Configuration
public class RouterFunctionConfig {

  @Bean
  public RouterFunction<?> helloRouterFunction() {
    return route(GET("/hello"),
        request -> ok().body(just("Hello World!"), String.class))
      ;
  }

}
```

The first thing to notice is that you've chosen to statically import a few helper classes that you can use to create the aforementioned functional types. You've also statically imported `Mono` to keep the rest of the code easier to read and understand.

In this `@Configuration` class, you have a single `@Bean` method of type `RouterFunction<?>`. As mentioned, a `RouterFunction` declares mappings between one or more `RequestPredicate` objects and the functions that will handle the matching request(s).

The `route()` method from `RouterFunctions` accepts two parameters: a `RequestPredicate` and a function to handle matching requests. In this case, the `GET()` method from `RequestPredicates` declares a `RequestPredicate` that matches HTTP `GET` requests for the /hello path.

As for the handler function, it's written as a lambda, although it can also be a method reference. Although it isn't explicitly declared, the handler lambda accepts a

`ServerRequest` as a parameter. It returns a `ServerResponse` using `ok()` from `ServerResponse` and `body()` from `BodyBuilder`, which was returned from `ok()`. This was done to create a response with an HTTP 200 (OK) status code and a body payload that says `"Hello World!"`

As written, the `helloRouterFunction()` method declares a `RouterFunction` that handles only a single kind of request. But if you need to handle a different kind of request, you don't have to write another `@Bean` method, although you can. You only need to call `andRoute()` to declare another `RequestPredicate` to function mapping. For example, here's how you might add another handler for `GET` requests for `/bye`:

```
@Bean
public RouterFunction<?> helloRouterFunction() {
  return route(GET("/hello"),
      request -> ok().body(just("Hello World!"), String.class))
      .andRoute(GET("/bye"),
      request -> ok().body(just("See ya!"), String.class))
  ;
}
```

Hello World samples are fine for dipping your toes into something new. But let's amp it up a bit and see how to use Spring's functional web programming model to handle requests that resemble real-world scenarios.

To demonstrate how the functional programming model might be used in a real-world application, let's reinvent the functionality of `TacoController` in the functional style. The following configuration class is a functional analog to `TacoController`:

```
package tacos.web.api;

import static org.springframework.web.reactive.function.server
     .RequestPredicates.GET;
import static org.springframework.web.reactive.function.server
     .RequestPredicates.POST;
import static org.springframework.web.reactive.function.server
     .RequestPredicates.queryParam;
import static org.springframework.web.reactive.function.server
     .RouterFunctions.route;

import java.net.URI;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.reactive.function.server.RouterFunction;
import org.springframework.web.reactive.function.server.ServerRequest;
import org.springframework.web.reactive.function.server.ServerResponse;

import reactor.core.publisher.Mono;
import tacos.Taco;
import tacos.data.TacoRepository;
```

```
@Configuration
public class RouterFunctionConfig {

  @Autowired
  private TacoRepository tacoRepo;

  @Bean
  public RouterFunction<?> routerFunction() {
    return route(GET("/api/tacos").
              and(queryParam("recent", t->t != null )),
              this::recents)
        .andRoute(POST("/api/tacos"), this::postTaco);
  }

  public Mono<ServerResponse> recents(ServerRequest request) {
    return ServerResponse.ok()
        .body(tacoRepo.findAll().take(12), Taco.class);
  }

  public Mono<ServerResponse> postTaco(ServerRequest request) {
    return request.bodyToMono(Taco.class)
        .flatMap(taco -> tacoRepo.save(taco))
        .flatMap(savedTaco -> {
            return ServerResponse
                .created(URI.create(
                    "http://localhost:8080/api/tacos/" +
                    savedTaco.getId()))
                .body(savedTaco, Taco.class);
        });
  }
}
```

As you can see, the `routerFunction()` method declares a `RouterFunction<?>` bean, like the Hello World example. But it differs in what types of requests are handled and how they're handled. In this case, the `RouterFunction` is created to handle `GET` requests for /api/tacos?recent and `POST` requests for /api/tacos.

What stands out even more is that the routes are handled by method references. Lambdas are great when the behavior behind a `RouterFunction` is relatively simple and brief. In many cases, however, it's better to extract that functionality into a separate method (or even into a separate method in a separate class) to maintain code readability.

For your needs, `GET` requests for /api/tacos?recent will be handled by the `recents()` method. It uses the injected `TacoRepository` to fetch a `Flux<Taco>`, from which it takes 12 items. It then wraps the `Flux<Taco>` in a `Mono<ServerResponse>` so that we can ensure that the response has an HTTP 200 (OK) status by calling `ok()` on the `ServerResponse`. It's important to understand that even though up to 12 tacos are returned, there is only one server response—that's why it is returned in a `Mono` and not a `Flux`. Internally, Spring will still stream the `Flux<Taco>` to the client as a `Flux`.

Meanwhile, `POST` requests for /api/tacos are handled by the `postTaco()` method, which extracts a `Mono<Taco>` from the body of the incoming `ServerRequest`. The

`postTaco()` method then uses a series of `flatMap()` operations to save that taco to the `TacoRepository` and create a `ServerResponse` with an HTTP 201 (CREATED) status code and the saved `Taco` object in the response body.

The `flatMap()` operations are used to ensure that at each step in the flow, the result of the mapping is wrapped in a `Mono`, starting with a `Mono<Taco>` after the first `flatMap()` and ultimately ending with a `Mono<ServerResponse>` that is returned from `postTaco()`.

12.3 Testing reactive controllers

When it comes to testing reactive controllers, Spring hasn't left us in the lurch. Indeed, Spring has introduced `WebTestClient`, a new test utility that makes it easy to write tests for reactive controllers written with Spring WebFlux. To see how to write tests with `WebTestClient`, let's start by using it to test the `recentTacos()` method from the `TacoController` that you wrote in section 12.1.2.

12.3.1 Testing GET requests

One thing we'd like to assert about the `recentTacos()` method is that if an HTTP `GET` request is issued for the path /api/tacos?recent, then the response will contain a JSON payload with no more than 12 tacos. The test class in the next listing is a good start.

Listing 12.1 Using `WebTestClient` to test `TacoController`

```
package tacos.web.api;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;
import java.util.ArrayList;
import java.util.List;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;
import org.springframework.http.MediaType;
import org.springframework.test.web.reactive.server.WebTestClient;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;
import tacos.Ingredient;
import tacos.Ingredient.Type;
import tacos.Taco;
import tacos.data.TacoRepository;

public class TacoControllerTest {

  @Test
  public void shouldReturnRecentTacos() {
    Taco[] tacos = {
        testTaco(1L), testTaco(2L),
        testTaco(3L), testTaco(4L),
        testTaco(5L), testTaco(6L),
        testTaco(7L), testTaco(8L),
```

Creates some test data

```
        testTaco(9L), testTaco(10L),
        testTaco(11L), testTaco(12L),
        testTaco(13L), testTaco(14L),
        testTaco(15L), testTaco(16L)};
    Flux<Taco> tacoFlux = Flux.just(tacos);

    TacoRepository tacoRepo = Mockito.mock(TacoRepository.class);
    when(tacoRepo.findAll()).thenReturn(tacoFlux);

    WebTestClient testClient = WebTestClient.bindToController(
        new TacoController(tacoRepo))
        .build();

    testClient.get().uri("/api/tacos?recent")
      .exchange()
      .expectStatus().isOk()
      .expectBody()
        .jsonPath("$").isArray()
        .jsonPath("$").isNotEmpty()
        .jsonPath("$[0].id").isEqualTo(tacos[0].getId().toString())
        .jsonPath("$[0].name").isEqualTo("Taco 1")
        .jsonPath("$[1].id").isEqualTo(tacos[1].getId().toString())
        .jsonPath("$[1].name").isEqualTo("Taco 2")
        .jsonPath("$[11].id").isEqualTo(tacos[11].getId().toString())
        .jsonPath("$[11].name").isEqualTo("Taco 12")
        .jsonPath("$[12]").doesNotExist();
  }

  ...

}
```

Mocks the TacoRepository

Creates a WebTestClient

Requests recent tacos

Verifies the expected response

The first thing that the `shouldReturnRecentTacos()` method does is set up test data in the form of a `Flux<Taco>`. This `Flux` is then provided as the return value from the `findAll()` method of a mock `TacoRepository`.

With regard to the `Taco` objects that will be published by `Flux`, they're created with a utility method named `testTaco()` that, when given a number, produces a `Taco` object whose ID and name are based on that number. The `testTaco()` method is implemented as follows:

```
private Taco testTaco(Long number) {
  Taco taco = new Taco();
  taco.setId(number != null ? number.toString(): "TESTID");
  taco.setName("Taco " + number);
  List<Ingredient> ingredients = new ArrayList<>();
  ingredients.add(
      new Ingredient("INGA", "Ingredient A", Type.WRAP));
  ingredients.add(
      new Ingredient("INGB", "Ingredient B", Type.PROTEIN));
  taco.setIngredients(ingredients);
  return taco;
}
```

For the sake of simplicity, all test tacos will have the same two ingredients. But their ID and name will be determined by the given number.

Meanwhile, back in the `shouldReturnRecentTacos()` method, you instantiated a `TacoController`, injecting the mock `TacoRepository` into the constructor. The controller is given to `WebTestClient.bindToController()` to create an instance of `WebTestClient`.

With all of the setup complete, you're now ready to use `WebTestClient` to submit a `GET` request to /api/tacos?recent and verify that the response meets your expectations. Calling `get().uri("/api/tacos?recent")` describes the request you want to issue. Then a call to `exchange()` submits the request, which will be handled by the controller that `WebTestClient` is bound to—the `TacoController`.

Finally, you can affirm that the response is as expected. By calling `expectStatus()`, you assert that the response has an HTTP 200 (OK) status code. After that, you see several calls to `jsonPath()` that assert that the JSON in the response body has the values it should have. The final assertion checks that the 12th element (in a zero-based array) is nonexistent, because the result should never have more than 12 elements.

If the JSON returns are complex, with a lot of data or highly nested data, it can be tedious to use `jsonPath()`. In fact, I left out many of the calls to `jsonPath()` in listing 12.1 to conserve space. For those cases where it may be clumsy to use `jsonPath()`, `WebTestClient` offers `json()`, which accepts a `String` parameter containing the JSON to compare the response against.

For example, suppose that you've created the complete response JSON in a file named recent-tacos.json and placed it in the classpath under the path /tacos. Then you can rewrite the `WebTestClient` assertions to look like this:

```
ClassPathResource recentsResource =
    new ClassPathResource("/tacos/recent-tacos.json");
String recentsJson = StreamUtils.copyToString(
    recentsResource.getInputStream(), Charset.defaultCharset());

testClient.get().uri("/api/tacos?recent")
  .accept(MediaType.APPLICATION_JSON)
  .exchange()
  .expectStatus().isOk()
  .expectBody()
    .json(recentsJson);
```

Because `json()` accepts a `String`, you must first load the classpath resource into a `String`. Thankfully, Spring's `StreamUtils` makes this easy with `copyToString()`. The `String` that's returned from `copyToString()` will contain the entire JSON you expect in the response to your request. Giving it to the `json()` method ensures that the controller is producing the correct output.

Another option offered by `WebTestClient` allows you to compare the response body with a list of values. The `expectBodyList()` method accepts either a `Class` or a `ParameterizedTypeReference` indicating the type of elements in the list and returns

a ListBodySpec object to make assertions against. Using expectBodyList(), you can rewrite the test to use a subset of the same test data you used to create the mock TacoRepository, as shown here:

```
testClient.get().uri("/api/tacos?recent")
  .accept(MediaType.APPLICATION_JSON)
  .exchange()
  .expectStatus().isOk()
  .expectBodyList(Taco.class)
    .contains(Arrays.copyOf(tacos, 12));
```

Here you assert that the response body contains a list that has the same elements as the first 12 elements of the original Taco array you created at the beginning of the test method.

12.3.2 Testing POST requests

WebTestClient can do more than just test GET requests against controllers. It can also be used to test any kind of HTTP method. Table 12.1 maps HTTP methods to WebTestClient methods.

Table 12.1 WebTestClient tests any kind of request against Spring WebFlux controllers.

HTTP method	WebTestClient method
GET	.get()
POST	.post()
PUT	.put()
PATCH	.patch()
DELETE	.delete()
HEAD	.head()

As an example of testing another HTTP method request against a Spring WebFlux controller, let's look at another test against TacoController. This time, you'll write a test of your API's taco creation endpoint by submitting a POST request to /api/tacos as follows:

```
@SuppressWarnings("unchecked")
@Test
public void shouldSaveATaco() {
  TacoRepository tacoRepo = Mockito.mock(
              TacoRepository.class);

  WebTestClient testClient = WebTestClient.bindToController(
      new TacoController(tacoRepo)).build();
```

Mocks the TacoRepository

Creates a WebTestClient

```
  Mono<Taco> unsavedTacoMono = Mono.just(testTaco(1L));
  Taco savedTaco = testTaco(1L);
  Flux<Taco> savedTacoMono = Flux.just(savedTaco);

  when(tacoRepo.saveAll(any(Mono.class))).thenReturn(savedTacoMono);


  testClient.post()
      .uri("/api/tacos")
      .contentType(MediaType.APPLICATION_JSON)
      .body(unsavedTacoMono, Taco.class)
      .exchange()
      .expectStatus().isCreated()
      .expectBody(Taco.class)
      .isEqualTo(savedTaco);
}
```

Sets up test data

POSTs a taco

Verifies the response

As with the previous test method, `shouldSaveATaco()` starts by mocking `TacoRepository`, building a `WebTestClient` that's bound to the controller, and setting up some test data. Then, it uses the `WebTestClient` to submit a `POST` request to /api/tacos, with a body of type `application/json` and a payload that's a JSON-serialized form of the `Taco` in the unsaved `Mono`. After performing `exchange()`, the test asserts that the response has an HTTP 201 (CREATED) status and a payload in the body equal to the saved `Taco` object.

12.3.3 Testing with a live server

The tests you've written so far have relied on a mock implementation of the Spring WebFlux framework so that a real server wouldn't be necessary. But you may need to test a WebFlux controller in the context of a server like Netty or Tomcat and maybe with a repository or other dependencies. That is to say, you may want to write an integration test.

To write a `WebTestClient` integration test, you start by annotating the test class with `@RunWith` and `@SpringBootTest` like any other Spring Boot integration test, as shown here:

```
package tacos;

import java.io.IOException;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.context.SpringBootTest.WebEnvironment;
import org.springframework.http.MediaType;
import org.springframework.test.context.junit.jupiter.SpringExtension;
import org.springframework.test.web.reactive.server.WebTestClient;

@ExtendWith(SpringExtension.class)
@SpringBootTest(webEnvironment=WebEnvironment.RANDOM_PORT)
public class TacoControllerWebTest {
```

```
  @Autowired
  private WebTestClient testClient;

}
```

By setting the `webEnvironment` attribute to `WebEnvironment.RANDOM_PORT`, you're asking Spring to start a running server listening on a randomly chosen port.[1]

You'll notice that you've also autowired a `WebTestClient` into the test class. This not only means that you'll no longer have to create one in your test methods but also that you won't need to specify a full URL when making requests. That's because the `WebTestClient` will be rigged to know which port the test server is running on. Now you can rewrite `shouldReturnRecentTacos()` as an integration test that uses the autowired `WebTestClient` as follows:

```
@Test
public void shouldReturnRecentTacos() throws IOException {
  testClient.get().uri("/api/tacos?recent")
    .accept(MediaType.APPLICATION_JSON).exchange()
    .expectStatus().isOk()
    .expectBody()
        .jsonPath("$").isArray()
        .jsonPath("$.length()").isEqualTo(3)
        .jsonPath("$[?(@.name == 'Carnivore')]").exists()
        .jsonPath("$[?(@.name == 'Bovine Bounty')]").exists()
        .jsonPath("$[?(@.name == 'Veg-Out')]").exists();
}
```

You've no doubt noticed that this new version of `shouldReturnRecentTacos()` has much less code. You no longer need to create a `WebTestClient` because you'll be making use of the autowired instance. And you don't have to mock `TacoRepository` because Spring will create an instance of `TacoController` and inject it with a real `TacoRepository`. In this new version of the test method, you use JSONPath expressions to verify values served from the database.

`WebTestClient` is useful when, in the course of a test, you need to consume the API exposed by a WebFlux controller. But what about when your application itself consumes some other API? Let's turn our attention to the client side of Spring's reactive web story and see how `WebClient` provides a REST client that deals in reactive types such as `Mono` and `Flux`.

12.4 Consuming REST APIs reactively

In chapter 8, you used `RestTemplate` to make client requests to the Taco Cloud API. `RestTemplate` is an old-timer, having been introduced in Spring version 3.0. In its time, it has been used to make countless requests on behalf of the applications that employ it.

[1] You could have also set `webEnvironment` to `WebEnvironment.DEFINED_PORT` and specified a port with the properties attribute, but that's generally inadvisable. Doing so opens the risk of a port clash with a concurrently running server.

But all of the methods provided by `RestTemplate` deal in nonreactive domain types and collections. This means that if you want to work with a response's data in a reactive way, you'll need to wrap it with a `Flux` or `Mono`. And if you already have a `Flux` or `Mono` and you want to send it in a `POST` or `PUT` request, then you'll need to extract the data into a nonreactive type before making the request.

It would be nice if there was a way to use `RestTemplate` natively with reactive types. Fear not. Spring offers `WebClient` as a reactive alternative to `RestTemplate`. `WebClient` lets you both send and receive reactive types when making requests to external APIs.

Using `WebClient` is quite different from using `RestTemplate`. Rather than having several methods to handle different kinds of requests, `WebClient` has a fluent builder-style interface that lets you describe and send requests. The general usage pattern for working with `WebClient` follows:

- Create an instance of `WebClient` (or inject a `WebClient` bean)
- Specify the HTTP method of the request to send
- Specify the URI and any headers that should be in the request
- Submit the request
- Consume the response

Let's look at several examples of `WebClient` in action, starting with how to use `WebClient` to send HTTP `GET` requests.

12.4.1 GETting resources

As an example of `WebClient` usage, suppose that you need to fetch an `Ingredient` object by its ID from the Taco Cloud API. Using `RestTemplate`, you might use the `getForObject()` method. But with `WebClient`, you build the request, retrieve a response, and then extract a `Mono` that publishes the `Ingredient` object, as shown here:

```
Mono<Ingredient> ingredient = WebClient.create()
    .get()
    .uri("http://localhost:8080/ingredients/{id}", ingredientId)
    .retrieve()
    .bodyToMono(Ingredient.class);

ingredient.subscribe(i -> { ... });
```

Here you create a new `WebClient` instance with `create()`. Then you use `get()` and `uri()` to define a `GET` request to http://localhost:8080/ingredients/{id}, where the {id} placeholder will be replaced by the value in `ingredientId`. The `retrieve()` method executes the request. Finally, a call to `bodyToMono()` extracts the response's body payload into a `Mono<Ingredient>` on which you can continue applying additional `Mono` operations.

To apply additional operations on the `Mono` returned from `bodyToMono()`, it's important to subscribe to it before the request will even be sent. Making requests that

can return a collection of values is easy. For example, the following snippet of code fetches all ingredients:

```
Flux<Ingredient> ingredients = WebClient.create()
    .get()
    .uri("http://localhost:8080/ingredients")
    .retrieve()
    .bodyToFlux(Ingredient.class);

ingredients.subscribe(i -> { ... });
```

For the most part, fetching multiple items is the same as making a request for a single item. The big difference is that instead of using `bodyToMono()` to extract the response's body into a `Mono`, you use `bodyToFlux()` to extract it into a `Flux`.

As with `bodyToMono()`, the `Flux` returned from `bodyToFlux()` hasn't yet been subscribed to. This allows additional operations (filters, maps, and so forth) to be applied to the `Flux` before the data starts flowing through it. Therefore, it's important to subscribe to the resulting `Flux`, or else the request will never even be sent.

Making requests with a base URI

You may find yourself using a common base URI for many different requests. In that case, it can be useful to create a `WebClient` bean with a base URI and inject it anywhere it's needed. Such a bean could be declared like this (in any `@Configuration`-annotated class):

```
@Bean
public WebClient webClient() {
  return WebClient.create("http://localhost:8080");
}
```

Then, anywhere you need to make requests using that base URI, the `WebClient` bean can be injected and used like this:

```
@Autowired
WebClient webClient;

public Mono<Ingredient> getIngredientById(String ingredientId) {
  Mono<Ingredient> ingredient = webClient
    .get()
    .uri("/ingredients/{id}", ingredientId)
    .retrieve()
    .bodyToMono(Ingredient.class);

  ingredient.subscribe(i -> { ... });
}
```

Because the `WebClient` had already been created, you're able to get right to work by calling `get()`. As for the URI, you need to specify only the path relative to the base URI when calling `uri()`.

TIMING OUT ON LONG-RUNNING REQUESTS

One thing that you can count on is that networks aren't always reliable or as fast as you'd expect them to be. Or maybe a remote server is sluggish in handling a request. Ideally, a request to a remote service will return in a reasonable amount of time. But if not, it would be great if the client didn't get stuck waiting on a response for too long.

To avoid having your client requests held up by a sluggish network or service, you can use the `timeout()` method from `Flux` or `Mono` to put a limit on how long you'll wait for data to be published. As an example, consider how you might use `timeout()` when fetching ingredient data, as shown in the next code sample:

```
Flux<Ingredient> ingredients = webclient
    .get()
    .uri("/ingredients")
    .retrieve()
    .bodyToFlux(Ingredient.class);

ingredients
  .timeout(Duration.ofSeconds(1))
  .subscribe(
      i -> { ... },
      e -> {
        // handle timeout error
      });
```

As you can see, before subscribing to the `Flux`, you called `timeout()`, specifying a duration of 1 second. If the request can be fulfilled in less than 1 second, then there's no problem. But if the request is taking longer than 1 second, it'll time-out, and the error handler given as the second parameter to `subscribe()` is invoked.

12.4.2 Sending resources

Sending data with `WebClient` isn't much different from receiving data. As an example, let's say that you have a `Mono<Ingredient>` and want to send a `POST` request with the `Ingredient` that's published by the `Mono` to the URI with a relative path of /ingredients. All you must do is use the `post()` method instead of `get()` and specify that the `Mono` is to be used to populate the request body by calling `body()` as follows:

```
Mono<Ingredient> ingredientMono = Mono.just(
    new Ingredient("INGC", "Ingredient C", Ingredient.Type.VEGGIES));

Mono<Ingredient> result = webClient
  .post()
  .uri("/ingredients")
  .body(ingredientMono, Ingredient.class)
  .retrieve()
  .bodyToMono(Ingredient.class);

result.subscribe(i -> { ... });
```

If you don't have a Mono or Flux to send, but instead have the raw domain object on hand, you can use bodyValue(). For example, suppose that instead of a Mono <Ingredient>, you have an Ingredient that you want to send in the request body, as shown next:

```
Ingredient ingredient = ...;

Mono<Ingredient> result = webClient
  .post()
  .uri("/ingredients")
  .bodyValue(ingredient)
  .retrieve()
  .bodyToMono(Ingredient.class);

result.subscribe(i -> { ... });
```

Instead of a POST request, if you want to update an Ingredient with a PUT request, you call put() instead of post() and adjust the URI path accordingly, like so:

```
Mono<Void> result = webClient
  .put()
  .uri("/ingredients/{id}", ingredient.getId())
  .bodyValue(ingredient)
  .retrieve()
  .bodyToMono(Void.class);

result.subscribe();
```

PUT requests typically have empty response payloads, so you must ask bodyToMono() to return a Mono of type Void. On subscribing to that Mono, the request will be sent.

12.4.3 Deleting resources

WebClient also allows the removal of resources by way of its delete() method. For example, the following code deletes an ingredient for a given ID:

```
Mono<Void> result = webClient
  .delete()
  .uri("/ingredients/{id}", ingredientId)
  .retrieve()
  .bodyToMono(Void.class);

result.subscribe();
```

As with PUT requests, DELETE requests don't typically have a payload. Once again, you return and subscribe to a Mono<Void> to send the request.

12.4.4 Handling errors

All of the WebClient examples thus far have assumed a happy ending; there were no responses with 400-level or 500-level status codes. Should either kind of error statuses be returned, WebClient will log the failure and move on without incident.

If you need to handle such errors, then a call to `onStatus()` can be used to specify how various HTTP status codes should be dealt with. `onStatus()` accepts two functions: a predicate function, which is used to match the HTTP status, and a function that, given a `ClientResponse` object, returns a `Mono<Throwable>`.

To demonstrate how `onStatus()` can be used to create a custom error handler, consider the following use of `WebClient` that aims to fetch an ingredient given its ID:

```
Mono<Ingredient> ingredientMono = webClient
    .get()
    .uri("http://localhost:8080/ingredients/{id}", ingredientId)
    .retrieve()
    .bodyToMono(Ingredient.class);
```

As long as the value in `ingredientId` matches a known ingredient resource, then the resulting `Mono` will publish the `Ingredient` object when it's subscribed to. But what would happen if there were no matching ingredient?

When subscribing to a `Mono` or `Flux` that might end in an error, it's important to register an error consumer as well as a data consumer in the call to `subscribe()` as follows:

```
ingredientMono.subscribe(
    ingredient -> {
      // handle the ingredient data
      ...
    },
    error-> {
      // deal with the error
      ...
    });
```

If the ingredient resource is found, then the first lambda (the data consumer) given to `subscribe()` is invoked with the matching `Ingredient` object. But if it isn't found, then the request responds with a status code of HTTP 404 (NOT FOUND), which results in the second lambda (the error consumer) being given by default a `WebClientResponseException`.

The biggest problem with `WebClientResponseException` is that it's rather nonspecific as to what may have gone wrong to cause the `Mono` to fail. Its name suggests that there was an error in the response from a request made by `WebClient`, but you'll need to dig into `WebClientResponseException` to know what went wrong. And in any event, it would be nice if the exception given to the error consumer were more domain-specific instead of `WebClient`-specific.

By adding a custom error handler, you can provide code that translates a status code to a `Throwable` of your own choosing. Let's say that you want a failed request for an ingredient resource to cause the `Mono` to complete in error with an `UnknownIngredientException`. You can add the following call to `onStatus()` after the call to `retrieve()` to achieve that:

```
Mono<Ingredient> ingredientMono = webClient
    .get()
    .uri("http://localhost:8080/ingredients/{id}", ingredientId)
    .retrieve()
    .onStatus(HttpStatus::is4xxClientError,
            response -> Mono.just(new UnknownIngredientException()))
    .bodyToMono(Ingredient.class);
```

The first argument in the `onStatus()` call is a predicate that's given an `HttpStatus` and returns `true` if the status code is one you want to handle. And if the status code matches, then the response will be returned to the function in the second argument to handle as it sees fit, ultimately returning a `Mono` of type `Throwable`.

In the example, if the status code is a 400-level status code (e.g., a client error), then a `Mono` will be returned with an `UnknownIngredientException`. This causes the `ingredientMono` to fail with that exception.

Note that `HttpStatus::is4xxClientError` is a method reference to the `is4xxClientError` method of `HttpStatus`. It's this method that will be invoked on the given `HttpStatus` object. If you want, you can use another method on `HttpStatus` as a method reference; or you can provide your own function in the form of a lambda or method reference that returns a `boolean`.

For example, you can get even more precise in your error handling, checking specifically for an HTTP 404 (NOT FOUND) status by changing the call to `onStatus()` to look like this:

```
Mono<Ingredient> ingredientMono = webClient
    .get()
    .uri("http://localhost:8080/ingredients/{id}", ingredientId)
    .retrieve()
    .onStatus(status -> status == HttpStatus.NOT_FOUND,
            response -> Mono.just(new UnknownIngredientException()))
    .bodyToMono(Ingredient.class);
```

It's also worth noting that you can have as many calls to `onStatus()` as you need to handle any variety of HTTP status codes that might come back in the response.

12.4.5 Exchanging requests

Up to this point, you've used the `retrieve()` method to signify sending a request when working with `WebClient`. In those cases, the `retrieve()` method returned an object of type `ResponseSpec`, through which you were able to handle the response with calls to methods such as `onStatus()`, `bodyToFlux()`, and `bodyToMono()`. Working with `ResponseSpec` is fine for simple cases, but it's limited in a few ways. If you need access to the response's headers or cookie values, for example, then `ResponseSpec` isn't going to work for you.

When `ResponseSpec` comes up short, you can try calling `exchangeToMono()` or `exchangeToFlux()` instead of `retrieve()`. The `exchangeToMono()` method returns a `Mono` of type `ClientResponse`, on which you can apply reactive operations to inspect

and use data from the entire response, including the payload, headers, and cookies. The `exchangeToFlux()` method works much the same way but returns a `Flux` of type `ClientResponse` for working with multiple data items in the response.

Before we look at what makes `exchangeToMono()` and `exchangeToFlux()` different from `retrieve()`, let's start by looking at how similar they are. The following snippet of code uses a `WebClient` and `exchangeToMono()` to fetch a single ingredient by the ingredient's ID:

```
Mono<Ingredient> ingredientMono = webClient
    .get()
    .uri("http://localhost:8080/ingredients/{id}", ingredientId)
    .exchangeToMono(cr -> cr.bodyToMono(Ingredient.class));
```

This is roughly equivalent to the next example that uses `retrieve()`:

```
Mono<Ingredient> ingredientMono = webClient
    .get()
    .uri("http://localhost:8080/ingredients/{id}", ingredientId)
    .retrieve()
    .bodyToMono(Ingredient.class);
```

In the `exchangeToMono()` example, rather than use the `ResponseSpec` object's `bodyToMono()` to get a `Mono<Ingredient>`, you get a `Mono<ClientResponse>` on which you can apply a flat-mapping function to map the `ClientResponse` to a `Mono<Ingredient>`, which is flattened into the resulting `Mono`.

Let's see what makes `exchangeToMono()` different from `retrieve()`. Let's suppose that the response from the request might include a header named `X_UNAVAILABLE` with a value of `true` to indicate that (for some reason) the ingredient in question is unavailable. And for the sake of discussion, suppose that if that header exists, you want the resulting `Mono` to be empty—to not return anything. You can achieve this scenario by adding another call to `flatMap()`, but now it's simpler with a `WebClient` call like this:

```
Mono<Ingredient> ingredientMono = webClient
    .get()
    .uri("http://localhost:8080/ingredients/{id}", ingredientId)
    .exchangeToMono(cr -> {
      if (cr.headers().header("X_UNAVAILABLE").contains("true")) {
        return Mono.empty();
      }
      return Mono.just(cr);
    })
    .flatMap(cr -> cr.bodyToMono(Ingredient.class));
```

The new `flatMap()` call inspects the given `ClientRequest` object's headers, looking for a header named `X_UNAVAILABLE` with a value of `true`. If found, it returns an empty `Mono`. Otherwise, it returns a new `Mono` that contains the `ClientResponse`. In either event, the `Mono` returned will be flattened into the `Mono` that the next `flatMap()` call will operate on.

12.5 Securing reactive web APIs

For as long as there has been Spring Security (and even before that, when it was known as Acegi Security), its web security model has been built around servlet filters. After all, it just makes sense. If you need to intercept a request bound for a servlet-based web framework to ensure that the requester has proper authority, a servlet filter is an obvious choice. But Spring WebFlux puts a kink into that approach.

When writing a web application with Spring WebFlux, there's no guarantee that servlets are even involved. In fact, a reactive web application is debatably more likely to be built on Netty or some other nonservlet server. Does this mean that the servlet filter–based Spring Security can't be used to secure Spring WebFlux applications?

It's true that using servlet filters isn't an option when securing a Spring WebFlux application. But Spring Security is still up to the task. Starting with version 5.0.0, you can use Spring Security to secure both servlet-based Spring MVC and reactive Spring WebFlux applications. It does this using Spring's `WebFilter`, a Spring-specific analog to servlet filters that doesn't demand dependence on the servlet API.

What's even more remarkable, though, is that the configuration model for reactive Spring Security isn't much different from what you saw in chapter 4. In fact, unlike Spring WebFlux, which has a separate dependency from Spring MVC, Spring Security comes as the same Spring Boot security starter, regardless of whether you intend to use it to secure a Spring MVC web application or one written with Spring WebFlux. As a reminder, here's what the security starter looks like:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

That said, a few small differences exist between Spring Security's reactive and nonreactive configuration models. It's worth taking a quick look at how the two configuration models compare.

12.5.1 Configuring reactive web security

As a reminder, configuring Spring Security to secure a Spring MVC web application typically involves creating a new configuration class that extends `WebSecurityConfigurerAdapter` and is annotated with `@EnableWebSecurity`. Such a configuration class would override a `configuration()` method to specify web security specifics such as what authorizations are required for certain request paths. The following simple Spring Security configuration class serves as a reminder of how to configure security for a nonreactive Spring MVC application:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {
```